

《ARM Cortex-M3 嵌入式系统》

期末考试题库——参考答案与解析

(教师版)

试卷数量: 20 套

适用专业: 电子信息工程/自动化/物联网工程

考试范围: Cortex-M3 体系结构 + STM32 标准库 + GPIO/EXTI/USART/TIM/ADC

试卷 01 参考答案与解析

一、选择题 (15 分)

1. C 嵌入式系统资源通常受限, 不能任意扩展硬件。
2. B Cortex-M3 采用哈佛结构, 指令和数据总线分开。
3. C Cortex-M3 只有线程模式和处理模式, 没有系统模式 (ARM7/ARM9 才有)。
4. C Thumb-2 同时支持 16 位和 32 位指令, 兼顾代码密度和性能。
5. C STM32F103 系列采用 ARM Cortex-M3 内核。
6. B 推挽输出可输出高/低电平, 驱动能力较强。
7. C STM32F103 的 NVIC 支持 84 个中断源 (含 16 个系统异常)。
8. B GPIOA 挂载在 APB2 总线上, 使用 RCC_APB2PeriphClockCmd。
9. C USART 停止位可配置为 0.5/1/1.5/2 位。
10. A PWM1 模式: $CNT < CCR$ 时输出有效电平, $CNT \geq CCR$ 时输出无效电平。
11. B STM32F103 内置 12 位逐次逼近型 ADC。
12. B 同一编号的 EXTI 中断线 (如 EXTI0) 可连接任何一个 GPIO 端口的 Pin0, 但同一时刻只能选一个。
13. C BOOT0 和 BOOT1 引脚电平决定从 Flash/系统存储器/SRAM 启动。
14. B 位带操作将每个 bit 映射为别名区的一个 32 位字, 实现对单个比特的原子操作。
15. C GPIOx_CRL/CRH 寄存器用于配置引脚的模式和速度。

【解析】

选择题知识点分布：嵌入式系统概念 (1)、Cortex-M3 体系结构 (2,3,4)、STM32 型号 (5)、GPIO(6,15)、中断 (7)、时钟/总线 (8)、USART(9)、PWM(10)、ADC(11)、EXTI(12)、启动方式 (13)、位带操作 (14)。

常见错误选项分析：

- 第 3 题：系统模式是 ARM7/ARM9 的概念，Cortex-M3 取消了此模式——学生容易混淆。
- 第 7 题：240 是 NVIC 理论最大中断数，STM32F103 实际只实现了 84 个（含系统异常），需要区分。
- 第 12 题：学生容易误以为 EXTI0 只对应 PA0，实际上同一编号的所有端口 Pin 都可以连接到同一 EXTI 线，但同一时刻只能选一个。

二、填空题（15 分）

1. 嵌入式处理器；外围硬件设备；嵌入式操作系统；应用软件
2. 堆栈指针（SP）；链接寄存器（LR）；程序计数器（PC）
3. 8；4（输入模式：浮空/上拉/下拉/模拟）
4. 计数器寄存器（CNT）；预分频器（PSC）；自动重装载寄存器（ARR）
5. NVIC
6. 9600
7. 右对齐；左对齐
8. 上拉电阻
9. 24
10. 64KB/128KB/256KB/512KB（任一合理值）
11. 上升沿触发；下降沿触发；上升沿和下降沿均触发
12. 周期（频率）；占空比
13. GPIOx_IDR；GPIOx_ODR
14. 0.8（ $3.3\text{V}/4096 \approx 0.806\text{mV}$ ）
15. 抢占（preempt）；响应（sub）；优先级分组

三、简答题（10 分）

第 1 题（3 分）——得分点：

- (1) 位带操作的基本原理（1 分）：Cortex-M3 将 SRAM 区和外设区的两个 1MB 位带区中的每一位映射到别名区的一个 32 位字地址。对别名区该字的读写操作等价于对位带区位数据的原子操作。
- (2) 主要用途（2 分）：(a) 实现对单个 GPIO 引脚或寄存器位的原子置位/清零，避免“读-修改-写”过程中的竞态条件；(b) 简化位操作代码，提高效率。

第 2 题（3 分）——得分点：

- (1) PWM 基本原理（1.5 分）：PWM（Pulse Width Modulation）是通过调节脉冲信号的占空比来等效地传递能量或信息的技术。在固定频率下，改变高电平的持续时间（占空比），可以控制输出功率、亮度、转速等。
- (2) 参数调节（1.5 分）：改变 ARR（自动重载值）可调节 PWM 频率；改变 CCR（捕获/比较值）可调节占空比。频率 $f = f_{CK}/[(PSC + 1)(ARR + 1)]$ ，占空比 $= CCR/(ARR + 1)$ 。

第 3 题（4 分）——得分点：

- (1) 波特率（1 分）：每秒传输的码元（bit）数量，收发双方波特率需匹配。
- (2) 帧格式（3 分）：起始位（1bit，低电平）→ 数据位（5–9bit，LSB 先发）→ 校验位（可选，奇/偶/无）→ 停止位（0.5/1/1.5/2bit，高电平）。通过 USART_InitStructure 结构体配置。

四、程序设计题（60 分）

第 1 题（12 分）：GPIO LED 控制

【设计思路】

功能：按键按下 → LED 点亮，松开 → 熄灭。

思路：配置 PC13 推挽输出（LED），PA0 上拉输入（按键）。主循环中读取 PA0 状态：低电平（按下）→ PC13 输出低（LED 亮），高电平（松开）→ PC13 输出高（LED 灭）。

```
1 #include "stm32f10x.h"
2
3 void GPIO_Config(void) {
4     GPIO_InitTypeDef GPIO_InitStructure;
5     RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOC | RCC_APB2Periph_GPIOA, ENABLE);
6
7     /* PC13 LED - 推挽输出 */
8     GPIO_InitStructure.GPIO_Pin = GPIO_Pin_13;
9     GPIO_InitStructure.GPIO_Mode = GPIO_Mode_Out_PP;
10    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
11    GPIO_Init(GPIOC, &GPIO_InitStructure);
12    GPIO_SetBits(GPIOC, GPIO_Pin_13); /* 初始熄灭 */
13
14    /* PA0 按键 - 上拉输入 */
15    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_0;
16    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_IPU;
17    GPIO_Init(GPIOA, &GPIO_InitStructure);
18 }
19
20 int main(void) {
21     GPIO_Config();
22     while(1) {
23         if(GPIO_ReadInputDataBit(GPIOA, GPIO_Pin_0) == RESET) /* 按下低电平 */
24             GPIO_ResetBits(GPIOC, GPIO_Pin_13); /* LED亮 */
25         else
26             GPIO_SetBits(GPIOC, GPIO_Pin_13); /* LED灭 */
27     }
28 }
```

易错点：(1) 忘记使能 GPIO 时钟；(2) LED 低电平点亮时，逻辑与直觉相反；(3) 按键按下为低电平，GPIO_ReadInputDataBit 返回 RESET(0)。实际项目中建议加入消抖处理。

第 2 题（16 分）：USART 中断收发通信

【设计思路】

波特率计算：USART1 挂载在 APB2（72MHz）。 $BRR = 72000000 / (16 \times 115200) = 39.0625$ 。整数部分 39（0x27），小数部分 $0.0625 \times 16 = 1$ （0x1）。 $BRR = 39 \ll 4 \mid 1 = 0x271$ 。

```
1 #include "stm32f10x.h"
2
3 void USART1_Config(void) {
4     GPIO_InitTypeDef GPIO_InitStructure;
5     USART_InitTypeDef USART_InitStructure;
6     NVIC_InitTypeDef NVIC_InitStructure;
7
8     RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOA | RCC_APB2Periph_USART1, ENABLE);
9
10    /* PA9-TX 复用推挽输出 */
11    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_9;
12    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF_PP;
13    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
14    GPIO_Init(GPIOA, &GPIO_InitStructure);
15    /* PA10-RX 浮空输入 */
16    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_10;
17    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_IN_FLOATING;
18    GPIO_Init(GPIOA, &GPIO_InitStructure);
19
20    USART_InitStructure.USART_BaudRate = 115200;
21    USART_InitStructure.USART_WordLength = USART_WordLength_8b;
22    USART_InitStructure.USART_StopBits = USART_StopBits_1;
23    USART_InitStructure.USART_Parity = USART_Parity_No;
24    USART_InitStructure.USART_HardwareFlowControl = USART_HardwareFlowControl_None;
25    USART_InitStructure.USART_Mode = USART_Mode_Rx | USART_Mode_Tx;
26    USART_Init(USART1, &USART_InitStructure);
27
28    USART_ITConfig(USART1, USART_IT_RXNE, ENABLE); /* 使能接收中断 */
29
30    NVIC_InitStructure.NVIC_IRQChannel = USART1_IRQn;
31    NVIC_InitStructure.NVIC_IRQChannelPreemptionPriority = 1;
32    NVIC_InitStructure.NVIC_IRQChannelSubPriority = 0;
33    NVIC_InitStructure.NVIC_IRQChannelCmd = ENABLE;
34    NVIC_Init(&NVIC_InitStructure);
35
36    USART_Cmd(USART1, ENABLE);
37 }
38
39 /* USART1中断服务函数 */
40 void USART1_IRQHandler(void) {
41     uint8_t data;
42     if(USART_GetITStatus(USART1, USART_IT_RXNE) != RESET) {
43         data = (uint8_t)USART_ReceiveData(USART1); /* 读取数据自动清除RXNE */
44         USART_SendData(USART1, data); /* 回显 */
45         while(USART_GetFlagStatus(USART1, USART_FLAG_TXE) == RESET); /* 等待发送完成 */
46     }
47 }
48
49 int main(void) {
50     USART1_Config();
```

关键点：(1) 读 USART_DR 会自动清除 RXNE 标志；(2) 发送前需等待 TXE 标志置位；(3) USART1 挂载在 APB2 (72MHz)，USART2/USART3 挂载在 APB1 (36MHz)，计算波特率时注意区分；(4) 中断服务函数名必须与启动文件中一致：USART1_IRQHandler。

第 3 题 (18 分)：PWM 呼吸灯

【设计思路】

PWM 参数计算：目标频率 $f_{PWM} = 1\text{kHz}$ 。TIM3 挂载在 APB1，APB1 时钟 36MHz，由于 APB1 预分频不为 1（通常为 2），TIM3 时钟 = $\text{APB1} \times 2 = 72\text{MHz}$ 。

$$f_{PWM} = 72\text{MHz} / [(PSC + 1)(ARR + 1)] = 1000\text{Hz}$$

取 $ARR = 999$ ，则 $PSC + 1 = 72\text{M} / (1000 \times 1000) = 72$ ，故 $PSC = 71$ 。

占空比 30%： $CCR = (ARR + 1) \times 30\% = 1000 \times 0.3 = 300$ 。

```

1 #include "stm32f10x.h"
2
3 void TIM3_PWM_Config(void) {
4     GPIO_InitTypeDef GPIO_InitStructure;
5     TIM_TimeBaseInitTypeDef TIM_TimeBaseStructure;
6     TIM_OCInitTypeDef TIM_OCInitStructure;
7
8     RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOA, ENABLE);
9     RCC_APB1PeriphClockCmd(RCC_APB1Periph_TIM3, ENABLE);
10
11     /* PA6 - TIM3 CH1 复用推挽输出 */
12     GPIO_InitStructure.GPIO_Pin = GPIO_Pin_6;
13     GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF_PP;
14     GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
15     GPIO_Init(GPIOA, &GPIO_InitStructure);
16
17     /* 时基: PSC=71, ARR=999, f=72M/(72*1000)=1kHz */
18     TIM_TimeBaseStructure.TIM_Prescaler = 71;
19     TIM_TimeBaseStructure.TIM_Period = 999;
20     TIM_TimeBaseStructure.TIM_ClockDivision = 0;
21     TIM_TimeBaseStructure.TIM_CounterMode = TIM_CounterMode_Up;
22     TIM_TimeBaseInit(TIM3, &TIM_TimeBaseStructure);
23
24     /* CH1 PWM1模式, 有效电平高 */
25     TIM_OCInitStructure.TIM_OCMode = TIM_OCMode_PWM1;
26     TIM_OCInitStructure.TIM_OutputState = TIM_OutputState_Enable;
27     TIM_OCInitStructure.TIM_Pulse = 0; /* 初始占空比0 */
28     TIM_OCInitStructure.TIM_OCPolarity = TIM_OCPolarity_High;
29     TIM_OC1Init(TIM3, &TIM_OCInitStructure);
30     TIM_OC1PreloadConfig(TIM3, TIM_OCPreload_Enable);
31
32     TIM_Cmd(TIM3, ENABLE);
33 }
34
35 void delay_ms(uint16_t ms) {
36     uint32_t i, j;
37     for(i = 0; i < ms; i++)
38         for(j = 0; j < 72000; j++); /* 72MHz下约1ms */
39 }
40
41 int main(void) {
42     int16_t duty = 0, dir = 1;
43     TIM3_PWM_Config();
44
45     while(1) {
46         TIM_SetCompare1(TIM3, duty);
47         duty += dir;
48         if(duty >= 999) dir = -1; /* 到100%后减小 */
49         if(duty <= 0) dir = 1; /* 到0%后增大 */
50         delay_ms(2); /* 渐变速度 */

```

关键参数推导: $f_{PWM} = \frac{72 \times 10^6}{(71 + 1) \times (999 + 1)} = \frac{72 \times 10^6}{72 \times 1000} = 1000\text{Hz}$

CCR 计算: $CCR = (ARR + 1) \times Duty\%$ 。ARR=999 时, CCR=300 对应 30% 占空比。

易错点: (1) TIM3 挂载在 APB1, 时钟使能用 RCC_APB1PeriphClockCmd; (2) APB1 最大 36MHz, 但定时器时钟在 APB1 预分频不为 1 时翻倍为 72MHz; (3) GPIO 模式必须设为 AF_PP。

第 4 题 (14 分): EXTI + 定时器综合应用

【设计思路】

程序流程图 (状态转换):

```
[等待按键] --下降沿中断--> [进入EXTI ISR: 禁用EXTI, 启动TIM2 (50ms)]
--> [TIM2 50ms到] --> [TIM2 ISR: 读取PA0确认仍为低?]
--是--> [翻转LED状态, 清除EXTI标志, 重新使能EXTI, 停TIM2]
--否--> [抖动, 清除EXTI标志, 重新使能EXTI, 停TIM2]
--> [回到等待按键]
```



```
1 #include "stm32f10x.h"
2
3 volatile uint8_t key_debounce = 0; /* 消抖标志 */
4
5 void GPIO_Config(void) {
6     GPIO_InitTypeDef g;
7     RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOA | RCC_APB2Periph_GPIOC |
8                             RCC_APB2Periph_AFIO, ENABLE);
9     /* PC13 LED 推挽输出 */
10    g.GPIO_Pin = GPIO_Pin_13; g.GPIO_Mode = GPIO_Mode_Out_PP;
11    g.GPIO_Speed = GPIO_Speed_50MHz;
12    GPIO_Init(GPIOC, &g); GPIO_SetBits(GPIOC, GPIO_Pin_13);
13    /* PA0 按键 上拉输入 */
14    g.GPIO_Pin = GPIO_Pin_0; g.GPIO_Mode = GPIO_Mode_IPU;
15    GPIO_Init(GPIOA, &g);
16 }
17
18 void EXTI_Config(void) {
19     EXTI_InitTypeDef e; NVIC_InitTypeDef n;
20     GPIO_EXTILineConfig(GPIO_PortSourceGPIOA, GPIO_PinSource0);
21     e.EXTI_Line = EXTI_Line0; e.EXTI_Mode = EXTI_Mode_Interrupt;
22     e.EXTI_Trigger = EXTI_Trigger_Falling;
23     e.EXTI_LineCmd = ENABLE;
24     EXTI_Init(&e);
25     n.NVIC_IRQChannel = EXTI0_IRQn;
26     n.NVIC_IRQChannelPreemptionPriority = 1;
27     n.NVIC_IRQChannelSubPriority = 1;
28     n.NVIC_IRQChannelCmd = ENABLE;
29     NVIC_Init(&n);
30 }
31
32 void TIM2_Config(void) {
33     TIM_TimeBaseInitTypeDef t; NVIC_InitTypeDef n;
34     RCC_APB1PeriphClockCmd(RCC_APB1Periph_TIM2, ENABLE);
35     /* 50ms: PSC=71, ARR=49999 => 72M/72/50000=20Hz (50ms) */
36     t.TIM_Prescaler = 71;
37     t.TIM_Period = 49999;
38     t.TIM_ClockDivision = 0;
39     t.TIM_CounterMode = TIM_CounterMode_Up;
40     TIM_TimeBaseInit(TIM2, &t);
41     TIM_ITConfig(TIM2, TIM_IT_Update, ENABLE);
42     n.NVIC_IRQChannel = TIM2_IRQn;
43     n.NVIC_IRQChannelPreemptionPriority = 1;
44     n.NVIC_IRQChannelSubPriority = 2;
45     n.NVIC_IRQChannelCmd = ENABLE;
46     NVIC_Init(&n);
47 }
48
49 /* EXTI0 中断: 按键按下 */
50 void EXTI0_IRQHandler(void) {
```

关键点：(1) 消抖策略：进入 EXTI ISR 后立即屏蔽中断线，启动 50ms 定时器，定时结束后确认按键状态，再重新使能；(2) 直接操作寄存器 EXTI->IMR 来实现动态屏蔽/使能；(3) AFIO 时钟必须使能（配置 EXTI 端口映射需要）；(4) GPIO_ReadOutputDataBit 读取的是 ODR 值（实际输出状态），而非引脚实际电平。

试卷 02 参考答案与解析

一、选择题

1. C 嵌入式系统专用性强，通用性弱于桌面计算机。
2. B NVIC (Nested Vectored Interrupt Controller) 负责中断管理。
3. D LCD 显示屏不是最小系统必需组件。
4. B 开漏输出只能输出低电平，高电平需外接上拉电阻。
5. B TIM2-TIM7 挂载在 APB1 总线。
6. C AFIO_EXTICR 寄存器组用于选择 EXTI 对应的 GPIO 端口。
7. B USART 数据低位先发 (LSB First)。
8. A 占空比 = 高电平时间 / 周期 $\times 100\%$ 。
9. A $V_{in} = D \times V_{ref} / 2^N$ (注意分母是 2^N 不是 $2^N - 1$, 因为 0 也有意义)。
10. C 系统复位后 GPIO 默认为浮空输入模式，引脚电平不确定。
11. A NVIC_PriorityGroup_2: 抢占 2 位 (0-3)，响应 2 位 (0-3)。
12. D PCIe 是 PC 总线，STM32 不支持。
13. A Flash 主要用于存放程序代码和只读常量。
14. A 输入捕获在检测到边沿时锁存当前 CNT 值到 CCR。
15. A EXTI_ClearITPendingBit() 是标准库函数。

二、填空题

1. Flash、SRAM 及外设接口
2. 32; 32
3. 推挽输出; 开漏输出
4. 同步; 异步; 异步
5. 有效
6. 用户 Flash (主闪存)
7. 高; 慢
8. Pin0 至 Pin15

9. GPIO_InitTypeDef
10. 向上计数; ARR
11. 2
12. ADC_GetFlagStatus(); ADC 中断
13. USART_IT_RXNE
14. GPIOx_BSRR; 原子
15. 指令存储器; 数据存储器; 总线

三、简答题

第 1 题 (3 分) ——启动模式:

- (1) BOOT0=0, BOOT1=X: 从用户 Flash 启动 (0x08000000 映射到 0x00000000) ——正常运行时使用。
- (2) BOOT0=1, BOOT1=0: 从系统存储器启动 (0x1FFF0000 映射到 0x00000000) ——ISP 串口下载。
- (3) BOOT0=1, BOOT1=1: 从内嵌 SRAM 启动 (0x20000000 映射到 0x00000000) ——调试用。

第 2 题 (3 分) ——输出比较与 PWM: 区别: 输出比较 (Output Compare) 是当 CNT=CCR 时产生一次性的电平变化或中断; PWM 输出是持续产生固定频率和可变占空比的脉冲波形。联系: PWM 模式是输出比较模式的一种特殊配置 (通过设置 OCxM 位为 PWM1/PWM2 实现), 两者的硬件基础相同。

第 3 题 (4 分) ——DMA 在 ADC 中的作用: DMA (直接存储器访问) 可在不占用 CPU 的情况下, 将 ADC 每次转换的结果自动搬运到指定的内存地址。在 ADC 多通道扫描模式中, 由于规则组只有一个公共数据寄存器 (ADC_DR), 若不用 DMA, 前一个通道的结果会被后一个覆盖。DMA 可将每个通道的数据自动存入数组中, 实现高效的多通道数据采集。

四、程序设计题

第 1 题 (12 分): 矩阵键盘扫描

```

1 #include "stm32f10x.h"
2
3 void KeyPad_GPIO_Init(void) {
4     GPIO_InitTypeDef g;
5     RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOB, ENABLE);
6
7     /* PB0--PB3 行输出(推挽) */
8     g.GPIO_Pin = GPIO_Pin_0 | GPIO_Pin_1 | GPIO_Pin_2 | GPIO_Pin_3;
9     g.GPIO_Mode = GPIO_Mode_Out_PP;
10    g.GPIO_Speed = GPIO_Speed_50MHz;
11    GPIO_Init(GPIOB, &g);
12
13    /* PB4--PB7 列输入(上拉) */
14    g.GPIO_Pin = GPIO_Pin_4 | GPIO_Pin_5 | GPIO_Pin_6 | GPIO_Pin_7;
15    g.GPIO_Mode = GPIO_Mode_IPU;
16    GPIO_Init(GPIOB, &g);
17 }
18
19 int8_t KeyPad_Scan(void) {
20     uint8_t row, col;
21     const uint16_t rows[4] = {GPIO_Pin_0, GPIO_Pin_1,
22                                GPIO_Pin_2, GPIO_Pin_3};
23     const uint16_t cols[4] = {GPIO_Pin_4, GPIO_Pin_5,
24                                GPIO_Pin_6, GPIO_Pin_7};
25
26     for(row = 0; row < 4; row++) {
27         /* 当前行输出低, 其他行输出高 */
28         GPIOB->ODR = (GPIOB->ODR & 0xFFF0) | ~rows[row];
29         /* 其实更简洁: 所有行先全高, 当前行拉低 */
30         GPIO_ResetBits(GPIOB, rows[row]);
31         for(col = 0; col < 4; col++) {
32             if(GPIO_ReadInputDataBit(GPIOB, cols[col]) == RESET) {
33                 GPIO_SetBits(GPIOB, rows[row]); /* 恢复 */
34                 return row * 4 + col; /* 返回键值0--15 */
35             }
36         }
37         GPIO_SetBits(GPIOB, rows[row]); /* 恢复该行为高 */
38     }
39     return -1; /* 无按键 */
40 }

```

第 2 题 (16 分): 定时器中断 LED 闪烁

参数计算: TIM4 挂载 APB1, APB1 预分频不为 1 时 TIM4 时钟 = 72MHz。需要 250ms 中断周期:

$f_{INT} = 72\text{MHz}/[(PSC + 1)(ARR + 1)] = 4\text{Hz}$ (250ms 周期)

取 $PSC = 7199$, 则 $ARR + 1 = 72\text{M}/(7200 \times 4) = 2500$, $ARR = 2499$ 。即: $f_{CK_CNT} = 72\text{MHz}/7200 = 10\text{kHz}$, 计 2500 次 = 250ms。

```

1  #include "stm32f10x.h"
2
3  volatile uint8_t led_state = 0;
4
5  void GPIO_Config(void) {
6      GPIO_InitTypeDef g;
7      RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOC, ENABLE);
8      g.GPIO_Pin = GPIO_Pin_13; g.GPIO_Mode = GPIO_Mode_Out_PP;
9      g.GPIO_Speed = GPIO_Speed_50MHz;
10     GPIO_Init(GPIOC, &g); GPIO_SetBits(GPIOC, GPIO_Pin_13);
11 }
12
13 void TIM4_Config(void) {
14     TIM_TimeBaseInitTypeDef t; NVIC_InitTypeDef n;
15     RCC_APB1PeriphClockCmd(RCC_APB1Periph_TIM4, ENABLE);
16     t.TIM_Prescaler = 7199; /* 72MHz/7200=10kHz */
17     t.TIM_Period = 2499; /* 10kHz/2500=4Hz (250ms) */
18     t.TIM_ClockDivision = 0;
19     t.TIM_CounterMode = TIM_CounterMode_Up;
20     TIM_TimeBaseInit(TIM4, &t);
21     TIM_ITConfig(TIM4, TIM_IT_Update, ENABLE);
22     n.NVIC_IRQChannel = TIM4_IRQn;
23     n.NVIC_IRQChannelPreemptionPriority = 1;
24     n.NVIC_IRQChannelSubPriority = 0;
25     n.NVIC_IRQChannelCmd = ENABLE;
26     NVIC_Init(&n);
27     TIM_Cmd(TIM4, ENABLE);
28 }
29
30 void TIM4_IRQHandler(void) {
31     if(TIM_GetITStatus(TIM4, TIM_IT_Update) != RESET) {
32         TIM_ClearITPendingBit(TIM4, TIM_IT_Update);
33         led_state = !led_state;
34         if(led_state)
35             GPIO_ResetBits(GPIOC, GPIO_Pin_13); /* 亮 */
36         else
37             GPIO_SetBits(GPIOC, GPIO_Pin_13); /* 灭 */
38     }
39 }
40
41 int main(void) { GPIO_Config(); TIM4_Config(); while(1); }

```

第 3 题 (18 分): PWM 直流电机调速

参数计算: $f_{PWM} = 10\text{kHz}$, TIM3 时钟 = 72MHz (APB1 预分频 $\neq 1$ 时)。 $f_{PWM} = 72\text{MHz}/[(PSC + 1)(ARR + 1)] = 10\text{kHz}$

取 $PSC = 0$ (不分频), 则 $ARR + 1 = 72\text{M}/10\text{k} = 7200$, $ARR = 7199$ 。若取 $PSC = 71$, 则 $ARR + 1 = 72\text{M}/(72 \times 10\text{k}) = 100$, $ARR = 99$ 。

择 $PSC = 71, ARR = 7199$ (更好, 分辨率 7200 级)。占空比 40% 时: $CCR = 7200 \times 0.4 = 2880$ 。

```

1 #include "stm32f10x.h"
2
3 volatile uint16_t duty_percent = 50; /* 初始50% */
4
5 void PWM_Config(void) {
6     GPIO_InitTypeDef g; TIM_TimeBaseInitTypeDef t; TIM_OCInitTypeDef oc;
7     RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOA, ENABLE);
8     RCC_APB1PeriphClockCmd(RCC_APB1Periph_TIM3, ENABLE);
9
10    g.GPIO_Pin = GPIO_Pin_7; g.GPIO_Mode = GPIO_Mode_AF_PP;
11    g.GPIO_Speed = GPIO_Speed_50MHz;
12    GPIO_Init(GPIOA, &g);
13
14    t.TIM_Prescaler = 71; /* 72MHz/72=1MHz */
15    t.TIM_Period = 7199; /* 1MHz/7200=138.89Hz(实际约139Hz) */
16    /* 更好: PSC=0, ARR=7199 -> f=72M/7200=10kHz */
17    /* 这里纠正: 取PSC=0, ARR=7199 */
18    TIM_TimeBaseInit(TIM3, &t);
19
20    oc.TIM_OCMode = TIM_OCMode_PWM1;
21    oc.TIM_OutputState = TIM_OutputState_Enable;
22    oc.TIM_Pulse = 3600; /* 初始50%: 7200*0.5=3600 */
23    oc.TIM_OCPolarity = TIM_OCPolarity_High;
24    TIM_OC2Init(TIM3, &oc);
25    TIM_OC2PreloadConfig(TIM3, TIM_OCPreload_Enable);
26    TIM_Cmd(TIM3, ENABLE);
27 }
28
29 /* 正确版本: PSC=0, ARR=7199, f=72M/7200=10kHz */
30 void PWM_Config_Corrected(void) {
31     /* ...同上但 t.TIM_Prescaler = 0; t.TIM_Period = 7199; */
32 }
33
34 int main(void) {
35     uint8_t key1, key2;
36     PWM_Config_Corrected();
37     /* 按键GPIO初始化省略 */
38     while(1) {
39         key1 = GPIO_ReadInputDataBit(GPIOA, GPIO_Pin_0); /* + */
40         key2 = GPIO_ReadInputDataBit(GPIOA, GPIO_Pin_1); /* - */
41         if(key1 == RESET) {
42             duty_percent += 5;
43             if(duty_percent > 100) duty_percent = 100;
44             TIM_SetCompare2(TIM3, (uint16_t)(7200 * duty_percent / 100));
45             while(GPIO_ReadInputDataBit(GPIOA, GPIO_Pin_0) == RESET); /* 等释放 */
46         }
47         if(key2 == RESET) {
48             duty_percent -= 5;
49             if(duty_percent < 0) duty_percent = 0;
50             TIM_SetCompare2(TIM3, (uint16_t)(7200 * duty_percent / 100));

```


PWM 参数推导：取 $PSC = 0$ （不预分频），则 $f_{CK_CNT} = 72\text{MHz}$ 。 $f_{PWM} = 72\text{M}/(ARR+1) = 10\text{kHz} \Rightarrow ARR = 7199$ 。

占空比 40% 对应 $CCR2 = 7200 \times 0.4 = 2880$ 。

易错点：PSC=0 和不分频的概念容易混淆；TIM_SetCompare2 用于 CH2（PA7）。

第 4 题（14 分）：ADC 光敏 + PWM 自动调光

设计思路：环境光越暗 $\rightarrow$ 光敏电阻分压越高 $\rightarrow$ ADC 值越大。LED 需要补光（越暗越亮）。因此 PWM 占空比与 ADC 值成反比： $Duty = 100\% - ADC_Value/4095$ 。

电压换算： $V_{in} = ADC_Value \times 3.3/4095$ 。ADC 读数为 2048 时， $V_{in} = 2048 \times 3.3/4095 \approx 1.65\text{V}$ 。

```

1 #include "stm32f10x.h"
2
3 void ADC1_Config(void) {
4     ADC_InitTypeDef adc; GPIO_InitTypeDef g;
5     RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOA | RCC_APB2Periph_ADC1, ENABLE);
6     RCC_ADCCLKConfig(RCC_PCLK2_Div6); /* ADCCLK=72M/6=12MHz */
7
8     g.GPIO_Pin = GPIO_Pin_1; g.GPIO_Mode = GPIO_Mode_AIN;
9     GPIO_Init(GPIOA, &g);
10
11     ADC_InitStructure.ADC_Mode = ADC_Mode_Independent;
12     ADC_InitStructure.ADC_ScanConvMode = DISABLE;
13     ADC_InitStructure.ADC_ContinuousConvMode = DISABLE;
14     ADC_InitStructure.ADC_ExternalTrigConv = ADC_ExternalTrigConv_None;
15     ADC_InitStructure.ADC_DataAlign = ADC_DataAlign_Right;
16     ADC_InitStructure.ADC_NbrOfChannel = 1;
17     ADC_Init(ADC1, &ADC_InitStructure);
18
19     ADC_RegularChannelConfig(ADC1, ADC_Channel_1, 1, ADC_SampleTime_55Cycles5);
20     ADC_Cmd(ADC1, ENABLE);
21     ADC_ResetCalibration(ADC1);
22     while(ADC_GetResetCalibrationStatus(ADC1));
23     ADC_StartCalibration(ADC1);
24     while(ADC_GetCalibrationStatus(ADC1));
25 }
26
27 uint16_t ADC_Read(void) {
28     ADC_SoftwareStartConvCmd(ADC1, ENABLE);
29     while(ADC_GetSoftwareStartConvStatus(ADC1));
30     return ADC_GetConversionValue(ADC1);
31 }
32
33 /* TIM3 PWM初始化同前 (1kHz, ARR=999) */
34
35 int main(void) {
36     uint16_t adc_val; uint16_t duty;
37     ADC1_Config();
38     TIM3_PWM_Config(); /* 同试卷1的PWM初始化 */
39
40     while(1) {
41         adc_val = ADC_Read();
42         /* 映射: ADC大(亮) -> PWM小; ADC小(暗) -> PWM大 */
43         duty = 999 - (uint16_t)((uint32_t)adc_val * 999 / 4095);
44         TIM_SetCompare1(TIM3, duty);
45
46         /* 电压换算: V_in = adc_val * 3.3 / 4095 (V) */
47         /* 占空比映射: Duty% = duty / 1000 * 100 */
48     }
49 }

```

ADC 参数：参考电压 3.3V，12 位分辨率， $V_{in} = 2048 \times 3.3/4095 \approx 1.65\text{V}$ 。此时环境较亮（中间值），PWM 占空比约 50%。**映射关系：**ADC 值增大 $\rightarrow$ 环境变亮 $\rightarrow$ LED 应调暗 $\rightarrow$ 占空比减小。
公式： $Duty = 100\% - ADC/4095 \times 100\%$ 。

试卷 03 参考答案与解析

一、选择题

1. **B** 交叉编译是在一种平台（宿主机）上编译出在另一种平台（目标机）上运行的程序。
2. **C** Cortex-M3 硬件自动完成压栈/出栈和向量表查找，但不涉及操作系统的任务调度。
3. **B** NRST 是复位引脚，低电平有效。
4. **B** IPU（Input Pull-Up）= 上拉输入。
5. **A** USART_DR 是数据寄存器（读写对应不同的物理寄存器）。
6. **A** $f_{CK_CNT} = 72M/72 = 1MHz$ ，1ms 需计 1000 次， $ARR = 999$ 。
7. **B** STM32F103 的 ADC 有 16 个外部通道 + 2 个内部通道（温度传感器和 VREFINT）。
8. **A** EXTI_IMR（Interrupt Mask Register）= 中断屏蔽寄存器。
9. **A** 位带区：0x20000000–0x200FFFFF（SRAM）和 0x40000000–0x400FFFFF（外设）。
10. **B** SWD 仅需 SWCLK 和 SWDIO 两根信号线。
11. **B** APB2 最高 72MHz。
12. **B** $ARR=999$ 时，计数周期为 $ARR+1=1000$ （从 0 到 999 共 1000 个状态）。
13. **D** ADC_RegularChannelConfig 的参数包括 ADCx, Channel, Rank, SampleTime, 不包括 GPIO 引脚号。
14. **B** GPIO_Speed 设置的是输出驱动电路的翻转速度（压摆率），不是实际输出频率。
15. **D** STM32 的低功耗模式为睡眠、停止、待机，没有”超频模式”。

二、填空题

1. ARMv7-M; Thumb-2
2. 7（CRL, CRH, IDR, ODR, BSRR, BRR, LCKR）
3. 20; GPIO 引脚
4. 16; 8
5. TIMx_CCMR1/CCMR2
6. VDDA（3.3V）
7. 外设功能（如 USART/TIM/SPI 等）的推挽输出

8. USART_FLAG_TXE
9. 72; HSI; HSE; PLL
10. 24; 操作系统节拍; 简单延时
11. 72; 1
12. 4095
13. 清除
14. 硬件消抖; 软件消抖
15. Flash; 系统存储器; SRAM

三、简答题

第 1 题（3 分）——时钟树：HSI（内部 8MHz）和 HSE（外部 4–16MHz，通常 8MHz）可作为 PLL 的输入源。PLL 将输入倍频（2–16 倍）后输出 PLLCLK（最高 72MHz）。系统时钟 SYSCLK 可选 HSI、HSE 或 PLLCLK。SYSCLK 经 AHB 预分频 → HCLK（CPU/总线/DMA 时钟），再经 APB1（最高 36MHz）和 APB2（最高 72MHz）分频 → PCLK1/PCLK2。

第 2 题（3 分）——EXTI 配置流程：使能 GPIO 和 AFIO 时钟；配置 GPIO 为输入模式；调用 GPIO_EXTILineConfig() 选择 EXTI 线对应的 GPIO 端口；配置 EXTI_InitTypeDef（中断线、触发方式、中断/事件模式）；调用 EXTI_Init()；配置 NVIC（优先级分组、通道、抢占/响应优先级）；在 ISR 中检查并清除 EXTI 挂起位。

第 3 题（4 分）——ADC 转换四阶段：采样：闭合采样开关，输入电压对内部采样保持电容充电（时间可通过 ADC_SampleTime 配置）；保持：断开采样开关，电容保持采样结束时的电压；量化：逐次逼近寄存器（SAR）通过二分法逐步逼近输入电压值（12 位 ADC 需 12 个 ADC 时钟周期）；编码：将量化结果转换为 12 位二进制数字量输出到 ADC_DR。

四、程序设计题

第 1 题（12 分）：数码管显示

```
1 #include "stm32f10x.h"
2
3 /* 共阴数码管段码表 0--9 */
4 const uint8_t seg_table[10] = {
5     0x3F, /* 0: abcdef */
6     0x06, /* 1: bc */
7     0x5B, /* 2: abdeg */
8     0x4F, /* 3: abcdg */
9     0x66, /* 4: bcfg */
10    0x6D, /* 5: acdfg */
11    0x7D, /* 6: acdefg */
12    0x07, /* 7: abc */
13    0x7F, /* 8: abcdefg */
14    0x6F /* 9: abcdfg */
15 };
16
17 void GPIO_Config(void) {
18     GPIO_InitTypeDef g;
19     RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOA | RCC_APB2Periph_GPIOB, ENABLE);
20     g.GPIO_Pin = 0xFF; g.GPIO_Mode = GPIO_Mode_Out_PP;
21     g.GPIO_Speed = GPIO_Speed_50MHz;
22     GPIO_Init(GPIOA, &g); /* PA0--PA7: 段选 */
23     g.GPIO_Pin = GPIO_Pin_0 | GPIO_Pin_1 | GPIO_Pin_2 | GPIO_Pin_3;
24     GPIO_Init(GPIOB, &g); /* PB0--PB3: 位选 */
25     GPIO_SetBits(GPIOB, GPIO_Pin_0 | GPIO_Pin_1 | GPIO_Pin_2 | GPIO_Pin_3);
26 }
27
28 void DisplayDigit(uint8_t pos, uint8_t num) {
29     const uint16_t bit_sel[4] = {GPIO_Pin_0, GPIO_Pin_1,
30                                   GPIO_Pin_2, GPIO_Pin_3};
31     GPIO_SetBits(GPIOB, 0x0F); /* 全关 */
32     GPIO_Write(GPIOA, seg_table[num]); /* 送段码 */
33     GPIO_ResetBits(GPIOB, bit_sel[pos]); /* 开位选 */
34 }
35
36 int main(void) {
37     uint8_t pos = 0;
38     uint16_t num = 1234;
39     uint8_t digits[4] = {num/1000, (num/100)%10,
40                           (num/10)%10, num%10};
41     GPIO_Config();
42     while(1) {
43         DisplayDigit(pos, digits[pos]);
44         pos = (pos + 1) % 4;
45         /* 简易延时约2ms */
46         for(volatile uint32_t i=0; i<144000; i++);
47     }
48 }
```

第 2 题 (16 分): USART 指令控制 LED

核心代码 (USART1 中断服务函数):

```
1 void USART1_IRQHandler(void) {  
2     uint8_t ch;  
3     if(USART_GetITStatus(USART1, USART_IT_RXNE) != RESET) {  
4         ch = USART_ReceiveData(USART1);  
5         if(ch == '1') {  
6             GPIO_ResetBits(GPIOC, GPIO_Pin_13); /* 点亮LED */  
7             USART_SendData(USART1, 'O'); /* 回复OK */  
8         } else if(ch == '0') {  
9             GPIO_SetBits(GPIOC, GPIO_Pin_13); /* 熄灭LED */  
10            USART_SendData(USART1, 'K');  
11        } else {  
12            USART_SendData(USART1, '?');  
13        }  
14        while(USART_GetFlagStatus(USART1, USART_FLAG_TXE) == RESET);  
15    }  
16 }
```

第 3 题 (18 分): PWM 舵机角度控制

参数计算 (关键):

- PWM 周期 20ms $\rightarrow f_{PWM} = 50\text{Hz}$
- TIM2 时钟 = 72MHz (APB1 预分频 $\neq 1$ 时加倍)
- 取 $PSC = 71$: $f_{CK_CNT} = 72\text{M}/72 = 1\text{MHz}$ 。20ms 对应 $1\text{MHz} \times 0.02 = 20000$ 个计数。
- $ARR = 20000 - 1 = 19999$ 。验证: $f_{PWM} = 1\text{MHz}/20000 = 50\text{Hz}$ 。
- 0° (0.5ms): $CCR = 20000 \times 0.5/20 = 500$ 。
- 180° (2.5ms): $CCR = 20000 \times 2.5/20 = 2500$ 。
- 角度映射: $CCR = 500 + (angle/180) \times 2000$ 。

```
1  /* TIM2 PWM 舵机初始化 */
2  void TIM2_Servo_Init(void) {
3      /* ...GPIO: PA0 AF_PP... */
4      TIM_TimeBaseStructure.TIM_Prescaler = 71; /* 1MHz */
5      TIM_TimeBaseStructure.TIM_Period = 19999; /* 20ms */
6      /* ...其余同标准PWM初始化... */
7      TIM_OCInitStructure.TIM_Pulse = 1500; /* 初始90°(1.5ms) */
8  }
9
10 void SetServoAngle(uint8_t angle) {
11     uint16_t ccr;
12     if(angle > 180) angle = 180;
13     ccr = 500 + (uint32_t)angle * 2000 / 180;
14     TIM_SetCompare1(TIM2, ccr);
15 }
```

第 4 题（14 分）：USART 指令控制 PWM 综合


```
1 volatile uint8_t rx_buf[16], rx_idx = 0, rx_complete = 0;
2
3 void USART1_IRQHandler(void) {
4     uint8_t ch;
5     if(USART_GetITStatus(USART1, USART_IT_RXNE) != RESET) {
6         ch = USART_ReceiveData(USART1);
7         if(ch == '\r' || ch == '\n') {
8             rx_buf[rx_idx] = '\0';
9             rx_complete = 1;
10            rx_idx = 0;
11        } else if(rx_idx < 15) {
12            rx_buf[rx_idx++] = ch;
13        }
14    }
15 }
16
17 void ProcessCommand(void) {
18     uint16_t angle;
19     if(rx_buf[0] == 'A' && rx_complete) {
20         angle = (rx_buf[1]-'0')*100 + (rx_buf[2]-'0')*10 + (rx_buf[3]-'0');
21         SetServoAngle((uint8_t)angle); /* 使用第3题的函数 */
22         rx_complete = 0;
23     }
24 }
25
26 int main(void) {
27     USART1_Config(); /* 9600bps */
28     TIM2_Servo_Init();
29     while(1) {
30         if(rx_complete) ProcessCommand();
31     }
32 }
```

命令解析要点：(1) 接收到 “或” 标记帧结束；(2) 解析“A090”→提取数字部分→转换为整数；(3) 解析后的角度传递给 SetServoAngle 函数。

试卷 04 参考答案与解析

一、选择题

1. B 硬实时系统必须在规定时间内完成响应，否则系统失败。
2. B Cortex-M3 有 R0-R12 共 13 个通用寄存器。
3. C STM32 典型工作电压为 3.3V。
4. C 外设功能应使用复用功能输出（AF_PP 或 AF_OD）。
5. C USART 波特率误差应在 2% 以内（典型要求）。
6. B TIM_TimeBaseInit() 用于时基初始化。
7. D ADC 触发方式不包括串口触发。
8. B 在共享中断向量的 ISR 中通过检查 EXTI_PR 区分具体中断线。
9. A CONTROL[1]（SPSEL）控制使用 MSP(0) 还是 PSP(1)。
10. C TIM1 和 TIM8 是高级定时器。
11. C 数码管扫描频率一般不低于 50Hz 以避免闪烁。
12. A 抢占优先级高的中断可打断抢占优先级低的中断。
13. D Cortex-M3 没有 DivideError 异常（除零触发 UsageFault 或 HardFault）。
14. B AFIO 时钟主要用于配置 EXTI 端口映射和重映射功能。
15. C 1MHz 时钟，ARR=999，PWM 频率 = $1\text{MHz}/1000 = 1\text{kHz}$ 。

二、填空题

1. 嵌入式处理器（MCU/MPU）；ROM/Flash
2. 满递减；最后一个压入的数据
3. 上拉；下拉
4. 起始位；数据位；校验位（可选）；停止位
5. 无效
6. 规则组；注入组
7. EXTI_RTSR；EXTI_FTSR
8. RCC_APB2PeriphClockCmd；RCC_APB1PeriphClockCmd

9. 影子寄存器; ARPE
10. 存储器映射的起始地址; 4
11. $f_{CLK}/((PSC + 1) \times (ARR + 1))$
12. 1kHz; 1ms
13. 接收数据寄存器非空 (有数据可读)
14. 5-20; 延时再判断
15. 解锁; 上锁

三、简答题

第 1 题 (3 分) ——NVIC 优先级分组: 优先级分组决定了 4 位优先级寄存器中抢占优先级和响应优先级的位数分配。抢占优先级高的中断可打断抢占优先级低的中断 (嵌套); 响应优先级仅在抢占优先级相同时起作用 (同时挂起时响应优先级高的先响应), 不能打断。

第 2 题 (4 分) ——推挽 vs 开漏: 推挽输出: 由 P-MOS 和 N-MOS 交替导通, 输出高电平时 P-MOS 导通、N-MOS 截止, 驱动电流大, 适合驱动 LED 等。开漏输出: 只有 N-MOS, 只能输出低电平 (N-MOS 导通) 或高阻态 (N-MOS 截止), 高电平需外部上拉电阻, 适合 I2C 总线 (线与) 和电平转换。

第 3 题 (3 分) ——ADC 精度与校准: 采样时间越长, 输入阻抗对采样保持电容充电越充分, 转换精度越高, 但总转换时间增加。校准是为了消除 ADC 内部电容网络的制造偏差和温度漂移, 校准后 ADC 的线性度和精度都会提高。

四、程序设计题

第 1 题 (12 分): EXTI 按键中断计数

核心: EXTI0 ISR 中计数器加 1 并串口发送。USART1 初始化 (9600bps) + EXTI0 配置 (下降沿)。

第 2 题 (16 分): 定时器输入捕获测量频率

两次上升沿捕获的 CNT 差值 $\Delta = CCR_{new} - CCR_{old}$ 。频率 $f_{sig} = f_{CK_CNT}/\Delta = 72\text{MHz}/\Delta$ (无预分频时)。

第 3 题 (18 分): PWM 蜂鸣器音调控制

PSC/ARR 推导: $f_{CK} = 72\text{MHz}$ 。

$$PSC + 1 = \frac{72 \times 10^6}{(ARR + 1) \times f_{target}}$$

若约束 $ARR \in [500, 2000]$: - Do(523Hz): 取 $ARR = 999$, $PSC = \frac{72 \times 10^6}{1000 \times 523} - 1 \approx 137$ 。验证: $f = 72\text{M}/(138 \times 1000) \approx 522\text{Hz}$ (误差可接受)。

第 4 题 (14 分): ADC 温度采集 + PWM 风扇温控

温度转换: $T = 25 + (1.65 - V) \times 50$ 。 $V = ADC \times 3.3/4095$ 。控制策略: $T < 25^\circ\text{C} \rightarrow 10\%$; $25 \leq T \leq 40^\circ\text{C} \rightarrow$ 线性映射 $10\% \sim 100\%$; $T > 40^\circ\text{C} \rightarrow 100\%$ 。PWM 频率 25kHz: $PSC = 0$, $ARR = 72\text{M}/25\text{k} - 1 =$

2879。

试卷 05 参考答案与解析

一、选择题

1. B ARM 是 IP 核授权公司，不直接生产芯片。
2. D 硬件自动压栈只保存 R0–R3, R12, LR, PC, xPSR（共 8 个寄存器）。R4–R11 需要软件保存。
3. B HSE（High Speed External）= 外部高速晶振（4–16MHz）。
4. B 标准库正确的配置方式是填充结构体成员后调用 GPIO_Init()。
5. B $BRR = 72M / (16 \times 9600) \approx 468.75$ 。
6. A TIM_OCPolarity_High 表示有效电平为高电平。
7. B 规则组只有一个公共数据寄存器，多通道扫描时需 DMA。
8. A EXTI_SWIER（Software Interrupt Event Register）。
9. D SysTick 时钟源可选 AHB 时钟或 AHB/8（通过 CLKSOURCE 位选择）。
10. C 位带别名区中每个 bit 映射为一个 32 位字。
11. A $CCR = (ARR + 1) \times 25\% = 1000 \times 0.25 = 250$ 。
12. C 全双工（Full-Duplex）可同时双向传输。
13. A DMA 可自动更新 CCR 值而无需 CPU 干预。
14. B GPIO 速度等级越高，功耗和 EMI 越大，应根据需要选择合适的速度。
15. B 配置 EXTI 端口映射必须使能 AFIO 时钟。

二、填空题

1. 实时性
2. 特权级/用户级
3. 0x00000000; 0x00000004
4. GPIOx_IDR; 只读
5. TIM_TimeBaseInitTypeDef; TIM_OCInitTypeDef
6. ADC_ResetCalibration(); ADC_StartCalibration()
7. 0; 低; 1; 高
8. EXTI_SWIER

9. GPIO_ReadInputDataBit()
10. 偶数
11. 使能 ARR 的预装载（影子寄存器）
12. 999
13. 读-修改-写；竞态条件
14. EXTI9_5_IRQn; EXTI15_10_IRQn
15. 2; SWCLK; SWDIO

三、简答题

第 1 题（3 分）——线程/处理模式：线程模式是执行普通应用程序代码的模式，可在特权级或用户级运行；处理模式是执行异常/中断服务程序的模式，始终在特权级。复位后 CPU 处于特权级线程模式。

第 2 题（4 分）——PWM 配置步骤：使能 GPIO 和 TIM 时钟；配置 GPIO 为 AF_PP；配置 TIM_TimeBaseInitTypeDef（PSC, ARR, 计数方向等）并调用 TIM_TimeBaseInit(); 配置 TIM_OCInitTypeDef（PWM 模式、极性、CCR 初值等）并调用 TIM_OCxInit(); 使能 CCR 预装载 TIM_OCxPreloadConfig(); 调用 TIM_Cmd() 启动定时器。

第 3 题（3 分）——中断 vs 查询：中断方式 CPU 效率高（无需轮询），响应及时；但 ISR 应尽量简短，且需注意中断优先级和共享数据保护。查询方式简单但浪费 CPU 时间，适合对实时性要求不高的简单应用。

四、程序设计题

第 1 题（12 分）：GPIO 流水灯——使用 TIM4 定时中断（200ms），ISR 中更新 LED 状态，8 个 LED 依次移位。

第 2 题（16 分）：USART 发送字符串——使用 TXE 标志查询方式逐字节发送。int32_t 转十进制字符串通过除 10 取余实现。

第 3 题（18 分）：PWM RGB LED——三路 PWM（TIM3 CH1/CH2/CH3），频率 1kHz（ARR=999）。SetRGB(r,g,b) 函数分别写 CCR1/CCR2/CCR3。主循环按红 → 绿 → 蓝顺序渐变。

第 4 题（14 分）：EXTI 双按键 + 定时器 + PWM 综合
模式和状态管理核心代码：

```
1 volatile uint8_t mode = 0; /* 0=手动, 1=自动呼吸 */
2 volatile uint16_t duty = 0;
3 volatile int16_t breath_dir = 1;
4
5 /* EXTI ISR - K1模式切换 */
6 void EXTI0_IRQHandler(void) {
7     if(EXTI_GetITStatus(EXTI_Line0) != RESET) {
8         /* 20ms消抖后 */
9         mode = !mode;
10        USART_SendData(USART1, '0' + mode);
11        EXTI_ClearITPendingBit(EXTI_Line0);
12    }
13 }
14
15 /* EXTI ISR - K2参数调节 */
16 void EXTI1_IRQHandler(void) {
17     if(mode == 0) { /* 手动模式 */
18         duty = (duty + 100) % 1001; /* 步进10% */
19         TIM_SetCompare1(TIM3, duty);
20     }
21     EXTI_ClearITPendingBit(EXTI_Line1);
22 }
23
24 /* TIM2 ISR - 自动呼吸渐变 (10ms周期) */
25 void TIM2_IRQHandler(void) {
26     static uint16_t tick = 0;
27     if(TIM_GetITStatus(TIM2, TIM_IT_Update) != RESET) {
28         TIM_ClearITPendingBit(TIM2, TIM_IT_Update);
29         if(mode == 1) {
30             tick++;
31             if(tick >= 3) { /* 30ms更新一次, 3秒周期 */
32                 tick = 0;
33                 duty += breath_dir;
34                 if(duty >= 999) breath_dir = -1;
35                 if(duty <= 0) breath_dir = 1;
36                 TIM_SetCompare1(TIM3, duty);
37             }
38         }
39     }
40 }
```

试卷 06 参考答案与解析

一、选择题

1.C(桌面 OS 非嵌入式硬件) 2.A(Reset 优先级-3 最高) 3.B(3.3V) 4.B(上拉输入 IPU) 5.A(RTS 请求发送) 6.A($f_{CK_CNT} = f_{CLK}/(PSC + 1)$) 7.C(采样 + 逐次逼近) 8.A(EXTI0-4 各有独立向量) 9.B(16+32 位兼备) 10.C(高频需多等待周期) 11.B(逐行输出低逐列读) 12.A(查询清除标志) 13.B(频率与分辨率相互制约) 14.B(连续模式自动开始下一次) 15.B(使能/禁止中断源)

二、填空题

1. 专用性/资源受限 2. 特权级线程模式/主堆栈指针 (MSP) 3.HSI/HSE/LSE/LSI 4. 高/低 5.ARR/高 6.ADC_SampleTime 7. 发送数据寄存器为空/发送完成 8.EXTI0_IRQn/EXTI4_IRQn 9.AHB 时钟 (HCLK)/AHB 时钟/8 10.0x22000000/0x42000000 11. 翻转 12.1.65 13. 线与/I2C 等总线 14.GPIO 引脚/GND/内部上拉 15.Flash/SRAM

三、简答题

1.(3 分)GPIO 8 种模式：输入 4 种 (浮空/上拉/下拉/模拟)，输出 4 种 (推挽/开漏/复用推挽/复用开漏)。输入模式时输出驱动器禁止；模拟输入禁止施密特触发器以降低功耗。

2.(4 分)PSC/ARR/CCR 的意义：PSC(预分频器) 降低计数时钟频率 $= f_{CK}/(PSC+1)$ ；ARR(自动重载) 决定 PWM 周期 $= ARR+1$ 个计数周期；CCR(比较值) 决定占空比 $= CCR/(ARR+1)$ 。例： $f_{CK}=72\text{MHz}$ ，取 $PSC=71 \rightarrow f_{CK_CNT}=1\text{MHz}$ ，取 $ARR=499 \rightarrow f_{PWM}=2\text{kHz}$ ， $CCR=125 \rightarrow 25\%$ 。

3.(3 分)AFIO 作用：配置 EXTI 输入线的 GPIO 端口选择 (必须使能 AFIO 时钟)；外设重映射 (Remap)；JTAG/SWD 引脚重映射。

四、程序设计题

第 1 题 (12 分)：GPIO+ 定时器按键计数——TIM2 10ms 中断做按键扫描和消抖。计数器 uint8_t，通过 GPIO_Write(GPIOB, count) 将计数值显示在 8 个 LED 上。

第 2 题 (16 分)：USART 环形缓冲区


```

1 #define BUF_SIZE 64
2 typedef struct {
3     uint8_t buffer[BUF_SIZE];
4     volatile uint8_t head; /* 写指针 (ISR写) */
5     volatile uint8_t tail; /* 读指针 (main读) */
6 } RingBuf;
7
8 RingBuf rx_buf = {.head=0, .tail=0};
9
10 void USART1_IRQHandler(void) {
11     uint8_t next;
12     if(USART_GetITStatus(USART1, USART_IT_RXNE) != RESET) {
13         uint8_t ch = USART_ReceiveData(USART1);
14         next = (rx_buf.head + 1) % BUF_SIZE;
15         if(next != rx_buf.tail) { /* 非满 */
16             rx_buf.buffer[rx_buf.head] = ch;
17             rx_buf.head = next;
18         }
19     }
20 }
21
22 uint8_t RingBuf_GetChar(uint8_t *c) {
23     if(rx_buf.tail == rx_buf.head) return 0; /* 空 */
24     *c = rx_buf.buffer[rx_buf.tail];
25     rx_buf.tail = (rx_buf.tail + 1) % BUF_SIZE;
26     return 1;
27 }

```

第 3 题 (18 分): PWM 加热功率控制

参数计算: $f_{PWM} = 1\text{Hz}$ 。取 $PSC = 7199$, 则 $f_{CK_CNT} = 72\text{M}/7200 = 10\text{kHz}$ 。 $ARR + 1 = 10\text{k}/1 = 10000$, $ARR = 9999$ 。占空比 35%: $CCR3 = (9999 + 1) \times 0.35 = 3500$ 。

```

1 void SetHeaterPower(uint8_t percent) {
2     uint16_t ccr;
3     if(percent > 100) percent = 100;
4     ccr = (uint16_t)((uint32_t)percent * 10000) / 100;
5     TIM_SetCompare3(TIM3, ccr);
6 }

```

第 4 题 (14 分): ADC 多通道 + PWM 多路——ADC1 双通道扫描 (软件触发), 交替读取 CH1 和 CH2 结果。TIM3 两路 PWM(CH1:PA6, CH2:PA7)。ADC 值 $\rightarrow$ CCR 映射: $CCR = ADC_Value \times 999/4095$ 。

试卷 07 参考答案与解析

一、选择题

1.B(宿主机 = 开发 PC) 2.C(12 周期但可被延迟) 3.D(Arduino IDE 非调试器) 4.B(BSR 高 16 位写 1= 复位) 5.C(USART_Parity_Odd) 6.A(滤波毛刺) 7.A(2.4V VREF+ VDDA) 8.B(屏蔽对应中断线) 9.D(LSI 为低速时钟不可作为 SYSCLK) 10.B(防止运行中修改异常) 11.D(A 和 C 均正确) 12.B(AFIO_EXTICR) 13.A(TIM_OCpolarity 决定) 14.A(锁定直到复位) 15.B(标准外设库 = StdPeriph Library)

二、填空题

1.ARMv7-M/Thumb-2 2.0~7/8~15 3. 整数部分/小数部分 4.TIM_SetCounter()/TIM_GetCounter()
5.16/ADC_DR 6.20/PVD 输出 7.GPIO_StructInit() 8.CCR/(ARR+1) 9.0x20000000/20KB 等 10.
优先级分组 11.CCR 12. 连续转换模式/DMA 13. 自动清除 14. 延时 (如 20ms) 15.8MHz/72MHz

三、简答题

1.(4 分)**PWM 原理**: 时基单元 (CNT/PSC/ARR) 产生计数时钟和周期; 输出比较单元将 CNT 与 CCR 比较, 产生 OCxREF 信号; 极性选择决定有效电平; PWM 模式选择 (PWM1/PWM2) 决定 CNT 与 CCR 比较的输出逻辑。

2.(3 分) **规则组 vs 注入组**: 规则组最多 16 通道, 共用一个数据寄存器, 适合常规数据采集; 注入组最多 4 通道, 各有独立数据寄存器, 可打断规则组的转换序列, 适合紧急的模拟量采样 (如过流保护)。

3.(3 分) **优先级仲裁**: 同一抢占优先级下的多个中断同时到达时, 比较响应优先级 (数值小的优先); 若抢占和响应优先级都相同, 则比较中断向量号 (编号小的优先)。

四、程序设计题

第 1 题 (12 分): 矩阵键盘 + 数码管——4×4 键盘扫描 (行扫描法), 返回键值 0~15 映射到数码管段码 (0~9,A~F)。

第 2 题 (16 分): PWM 信号频率和占空比测量——TIM2 CH1(PA0) 上升沿捕获 → 记 T_1 , CH2(PA1) 下降沿捕获 → 记 T_2 , 下一个上升沿 → 记 T_3 。周期 = $T_3 - T_1$, 高电平时间 = $T_2 - T_1$ 。频率 $f = 72\text{M}/(T_3 - T_1)$ 。

第 3 题 (18 分): PWM 背光亮度调节——PSC=143, ARR=999: $f_{PWM} = 72\text{M}/(144 \times 1000) = 500\text{Hz}$ 。按键扫描调节 CCR(增量 = 50 = 5%)。USART 发送格式: "Brightness=XX%"。

第 4 题 (14 分): 三路 ADC+ 三路 PWM——ADC1 扫描 CH1/CH2/CH3(软件触发), 依次读取三路值。TIM3 三路 PWM(CH1/CH2/CH3), ARR=999。CCR_x=ADC_x×999/4095。ADC=4095 时 CCR=999(100% 占空比)。

试卷 08 参考答案与解析

一、选择题

1.B(MCU 集成 Flash/SRAM/外设) 2.B(PSP= 进程堆栈指针) 3.B(BOOT0=1,BOOT1=0=ISP)
4.B(IPD= 下拉输入) 5.D(GPIO 翻转非 USART 中断源) 6.A(更新事件 UEV) 7.B(保持电压稳定)
8.A(检查标志 → 处理 → 清除) 9.B(NVIC 使用 4 位优先级) 10.D(配置 GPIO 输出数据与时钟无关)
11.B(PWM2:CNT<CCR 输出无效) 12.D(Flash 擦除以页为单位, 擦除后为 1) 13.B(校验位由数据中
1 的个数决定) 14.B(239.5 个 ADC 时钟周期) 15.A(标准库底层操作寄存器)

二、填空题

1. 需求分析/体系结构设计/硬件软件协同设计/系统集成与测试 2.N/Z/C/V 3.RC 复位/低 4.4 5.
预分频器 (PSC)/计数器 (CNT)/自动重装载寄存器 (ARR) 6.12 7. 同时使能接收和发送 8.1 9.Sy-
sTick_CALIB/10ms 计数值 10.32/4 11.ARR+1 12.ADC_DR/DMA 13. 高 14.65535 15. 中断向量
表/堆栈初始化

三、简答题

1.(4 分) 推挽 vs 开漏电路: 推挽 =P-MOS(上)+N-MOS(下) 互补输出, 高低电平均有驱动能力; 开漏 = 仅 N-MOS 下拉, 高电平依赖外部上拉电阻。推挽适合 LED、普通 IO; 开漏适合 I2C(线与)、电平转换。
2.(3 分)ARPE 机制: ARPE=1 时, 对 ARR 的写入先存入影子寄存器, 在下一个更新事件 (UEV) 时才加载到实际寄存器。这保证了定时器运行时修改 ARR 不会产生异常的短周期或长周期。ARPE=0 时写入立即生效。
3.(3 分) 低功耗模式: 睡眠 (CPU 停, 外设运行, 任意中断/事件唤醒); 停止 (所有时钟停, 保持 SRAM, EXTI 线唤醒); 待机 (功耗最低, SRAM 丢失, 仅 WKUP/RTC/复位唤醒)。

四、程序设计题

第 1 题 (12 分): GPIO 中断 +LED 状态机——PA0-PA3 四个 EXTI(独立中断向量), TIM4 20ms 消抖。每个按键在 ISR 中翻转对应 LED。
第 2 题 (16 分): USART 不定长数据帧——帧状态机: IDLE→ 等 '\$'→RECEIVING(存数据, 遇 '#'则完成)→COMPLETE。最大 64 字节。
第 3 题 (18 分): PWM 双路电机正反转—— $f = 10\text{kHz}$, 取 $PSC = 71, ARR = 99$: $f_{PWM} = 72\text{M}/(72 \times 100) = 10\text{kHz}$ 。MotorControl(speed): speed=75 时 $CCR1 = 75, CCR2 = 0$ 。
第 4 题 (14 分): ADC+PWM 闭环恒温控制—— $V = ADC \times 3.3/4095$ 。 $T = V \times 100^\circ\text{C}$ 。 $Duty = K_p \times (T_{target} - T)$, $K_p = 5$, 限幅 0-100%。 $ADC=1024 \rightarrow V = 0.825\text{V} \rightarrow T = 82.5^\circ\text{C} \rightarrow \text{温差} = -52.5 \rightarrow Duty < 0 \rightarrow \text{限幅为 } 0\%(\text{超温不加热})$ 。

试卷 09 参考答案与解析

一、选择题

1.D(x86 桌面版非嵌入式典型架构) 2.A(重定位向量表基址) 3.B(HSI=8MHz) 4.B(AF_OD= 复用开漏) 5.B(总线保持高电平 = 空闲) 6.B(TIM_Pulse=CCR 值) 7.B(内部温度传感器 =CH16) 8.A(事件产生脉冲触发外设) 9.B(满递减) 10.B(APB1 36MHz) 11.D(A 或 B 均可) 12.A(Pin/Speed/Mode) 13.B(电平转换 + 线与) 14.B(依次转换后停止) 15.B(输出低电平)

二、填空题

1. 程序存储器/数据存储器/同时 2. 当前使用的堆栈/线程模式 3. 负载电容 4.RCC 时钟/GPIO_Init()
5.DIR/TIM_CounterMode_Up/TIM_CounterMode_Down 6. 低 12 位 7. 溢出错误/噪声错误/帧错误
8.EXTI_PR 9. 别名区中一个 32 位字 10.72 11. 原子 12.DR 寄存器空/所有数据发送完毕 13.H
桥/两路互补 14. 定时器触发/外部触发 15.stm32f10x.h/stm32f10x_rcc.h 等

三、简答题

1.(3 分) 哈佛 vs 冯·诺依曼: 冯结构 = 指令和数据共用总线 (瓶颈), 哈结构 = 指令和数据总线分离 (可并行访问)。Cortex-M3 采用哈佛结构, 但对外呈现冯·诺依曼风格 (统一地址空间)。
2.(4 分)USART 中断接收流程: 检测起始位 (下降沿)→ 在数据位中心采样 (16 倍过采样)→ 移位寄存器逐位接收 → 接收完成 → 数据传入 RDR→ RXNE 标志置位 → 若 RXNEIE 使能则产生中断 → ISR 中读 DR 自动清除 RXNE。
3.(3 分) 边沿对齐 vs 中央对齐: 边沿对齐 =CNT 单向上计数 (通用定时器支持); 中央对齐 =CNT 先上后下双向计数 (通用定时器也支持, 需设置 CMS 位)。中央对齐谐波更小, 适合电机控制。

四、程序设计题

第 1 题 (12 分): GPIO 按键模式切换——状态机 4 个模式, TIM4 20ms 消抖扫描。
第 2 题 (16 分): PWM 捕获 + 频率占空比——双沿捕获测频和占空比 (同试卷 7 第 2 题)。
第 3 题 (18 分): PWM 色温控制——两路 PWM(CH1 暖白/CH2 冷白) 比例混合。SetColorTemp(mode) 根据模式设置 CCR1 和 CCR2。
第 4 题 (14 分): USART+ADC+PWM 远程监控
核心命令解析代码:

```
1 void ProcessCmd(void) {
2     uint16_t val;
3     if(strcmp((char*)cmd, "R") == 0) {
4         uint16_t adc = ADC_Read();
5         uint16_t mv = (uint32_t)adc * 3300 / 4095;
6         sprintf((char*)tx_buf, "V=%umV\r\n", mv);
7         UART_SendString((char*)tx_buf);
8     } else if(cmd[0] == 'D') {
9         val = (cmd[1]-'0')*100 + (cmd[2]-'0')*10 + (cmd[3]-'0');
10        if(val <= 1000) {
11            TIM_SetCompare1(TIM3, val);
12            duty_current = val;
13        }
14    } else if(strcmp((char*)cmd, "S") == 0) {
15        sprintf((char*)tx_buf, "Duty=%u/1000\r\n", duty_current);
16        UART_SendString((char*)tx_buf);
17    }
18 }
```

试卷 10 参考答案与解析

一、选择题

1.B(ROM/Flash) 2.D(USART1 中断 = 外部中断, 非系统异常) 3.C(BOOT0=1,BOOT1=1→SRAM 启动) 4.B(输出低电平) 5.B(低电平 1 位时间) 6.A(初始化 CH1 输出比较) 7.C(ADCCLK 14MHz) 8.C(EXTI_Trigger_Rising_Falling) 9.C(0x40000000-0x5FFFFFFF) 10.B(通用定时器) 11.B(PWM1+CCR=0→CNT 0 始终成立 → 无效电平) 12.C(高速驱动电流最大但 EMI 也大) 13.A(使能更新中断) 14.A($1 \times 4095/3.3 \approx 1241$) 15.B(时钟信号)

二、填空题

1. 宿主机/目标机 2. 系统异常/外部中断 3.2-16/72 4. 按位或 (|) 5.USART_DR/发送移位寄存器 6.4 7.ADC_GetConversionValue() 8.EXTI_ClearITPendingBit()/反复进入中断 9.LSI 10.1/2/4/8 11.0x22000000 12.25 13.USART_SendData()/USART_ReceiveData() 14. 启动文件/main() 15. 计数器上溢下溢/软件设置 UG 位

三、简答题

1.(3 分) 存储器映射: Code 区 (0x00000000-0x1FFFFFFF, 含 Flash/系统存储器); SRAM 区 (0x20000000-0x3FFFFFFF); 外设区 (0x40000000-0x5FFFFFFF); 系统控制区 (0xE0000000-0xFFFFFFFF, 含 NVIC/SysTick 等)。
2.(4 分)PWM 频率与分辨率关系: $f_{PWM} = f_{CK}/[(PSC+1)(ARR+1)]$, 分辨率位数 $=\log_2(ARR+1)$ 。 f_{CK} 固定时, ARR 越大 → 分辨率越高 → f_{PWM} 越低。需根据应用折中: 电机控制需高分辨率可降低频率; LED 调光频率需 >100Hz 以防闪烁。
3.(3 分) 中断嵌套条件: 抢占优先级更高的中断才能打断当前 ISR; 响应优先级仅用于仲裁同时到达的中断, 不能实现嵌套; PRIMASK/FAULTMASK 未屏蔽; 当前 ISR 未设置 BASEPRI 屏蔽对方。

四、程序设计题

第 1 题 (12 分): USART 查询收发——主循环中检查 RXNE→读取 → 判断大小写 → 转换 → 等待 TXE→发送。
第 2 题 (16 分): PWM+EXTI 调速——PA0 EXTI→duty+20%(循环), PA1 EXTI→duty-20%(循环)。CCR=duty%×(ARR+1)/100。
第 3 题 (18 分): PWM 可调频率和占空比
PSC 重配置推导: ARR 固定为 999(10 位分辨率), PSC 动态调节。 $f_{PWM} = 72M/[(PSC+1) \times 1000]$, 故 $PSC = 72000/f_{PWM} - 1$ 。例: $f = 2000\text{Hz} \rightarrow PSC = 35$ 。占空比 CCR=Duty×999/1000。
第 4 题 (14 分): ADC+PWM 智能路灯——四级阈值判断 (2.5V/1.5V/0.5V), 分段占空比控制 (100%/50%/10%/0%)。USART 每 2s 上报。

试卷 11 参考答案与解析

一、选择题

1.B(在规定时间内响应) 2.A(R0-R3,R12,LR,PC,xPSR) 3.C($8\text{MHz} \times 9 = 72\text{MHz}$) 4.C(浮空输入) 5.B(移位寄存器 $\rightarrow$ DR $\rightarrow$ RXNE 置位) 6.D(TIM_Pulse 属于 TIM_OCInitTypeDef) 7.A(注入组可打断规则组) 8.A(仅 RTSR 使能) 9.A(MCO 输出到 PA8) 10.C(误差 $< 2\%$ 即可) 11.A(输出有效电平) 12.B(I2C 需要线与) 13.A(从 0 递增到 ARR) 14.A(仅 8 位精度时读高 8 位) 15.B(高 16 位写 1= 输出低电平)

二、填空题

1. 永久存储在 ROM/Flash 中 2.4/256 3.64/20 4. 输入输出和最大速度/具体的工作模式 5. 数字滤波器(死区时钟) 6.HCLK/ADC 预分频器 7.4/4 8. 限流电阻/电流过大烧毁引脚 9.OCxPE/TIMx_CCMRx 10.CTS/RTS 11. 谐波失真更小 12.1.2 13. 下一次更新事件 14.GPIO_InitTypeDef 15.APB2/APB1

三、简答题

1.(4 分) 中断压栈/出栈: 压栈顺序 $= \text{xPSR} \rightarrow \text{PC} \rightarrow \text{LR} \rightarrow \text{R12} \rightarrow \text{R3} \rightarrow \text{R2} \rightarrow \text{R1} \rightarrow \text{R0}$ (SP 递减)。LR 被硬件设为 EXC_RETURN 值 ($0\text{FFFFFFF1} =$ 返回线程模式 MSP)。返回时 PC 加载 EXC_RETURN 触发出栈, 硬件自动恢复 8 个寄存器。

2.(3 分) 更新中断 (UIE)vs 比较中断 (CCxIE): UIE 在 CNT 上溢/下溢时触发, 用于定时/周期性任务; CCxIE 在 $\text{CNT} = \text{CCR}$ 时触发, 用于 PWM/输出比较/输入捕获。

3.(3 分) 标准库核心文件: startup_xxx.s(启动文件, 向量表, 堆栈初始化); system_stm32f10x.c(系统时钟初始化); stm32f10x.h(寄存器定义); stm32f10x_conf.h(外设库头文件集中包含); 外设驱动 (如 stm32f10x_gpio.c 等)。

四、程序设计题

第 1 题 (12 分): 多 LED 模式——TIM4 500ms 中断, 模式 0(流水灯) 和模式 1(全闪), PA0 EXTI 切换模式。

第 2 题 (16 分): ADC+USART 电压表——ADC1 连续采样, $V_{mV} = \text{ADC} \times 3300 / 4095$, sprintf 格式化后 USART 发送。

第 3 题 (18 分): PWM 比例阀控制—— $f = 200\text{Hz}$, 取 $PSC = 71$, $ARR = 4999$: $72\text{M} / (72 \times 5000) = 200\text{Hz}$ 。PA0(+)PA1(-) 调档位 (步进 5%)。PC13 LED 指示 ($> 50\%$ 亮)。

第 4 题 (14 分): 双定时器 +USART 综合系统时基实现:

```
1 volatile uint32_t sys_tick_ms = 0;
2
3 /* TIM2: 1ms时基 */
4 void TIM2_IRQHandler(void) {
5     if(TIM_GetITStatus(TIM2, TIM_IT_Update) != RESET) {
6         TIM_ClearITPendingBit(TIM2, TIM_IT_Update);
7         sys_tick_ms++;
8     }
9 }
10
11 /* 主循环中100ms上报 */
12 int main(void) {
13     uint32_t last_report = 0;
14     /* ...初始化... */
15     while(1) {
16         if(sys_tick_ms - last_report >= 100) {
17             last_report = sys_tick_ms;
18             sprintf((char*)buf, "Duty=%u/1000\r\n", duty);
19             UART_SendString((char*)buf);
20         }
21         if(rx_complete) {
22             if(rx_buf[0]=='P')
23                 duty = atoi((char*)&rx_buf[1]);
24             rx_complete = 0;
25         }
26     }
27 }
```


试卷 12 参考答案与解析

一、选择题

1.B(硬实时错过截止 = 系统失败) 2.B(LR=EXC_RETURN 特殊值) 3.B(LSE=32.768kHz) 4.B(输入模式 BSRR 影响上拉下拉) 5.A(IDLE 检测不定长帧结束) 6.B(72) 7.A(注入组最多 4 通道) 8.A(使用 GPIO_EXTI_LineConfig) 9.B(1 万~10 万次) 10.B(APB2 外设时钟) 11.B(有效电平 = 低) 12.B(旧数据被覆盖) 13.A(反转法更快) 14.A(检查中断 + 使能状态) 15.D(睡眠 = 下一句继续, 停止 = 下一句继续; 待机 = 复位)

二、填空题

1.ARMv7-M/32/32 2. 备份域/VBAT 3. 竞态条件/BSRR 4.DR 空时/移位寄存器也空 5. $f_{CK}/[(P+1)(A+1)]$ 6. 线性度 7.PB0 8.GPIO_Mode_AIN 9.TIMx_CR1 的 CKD 位 10.11 11.PWM1/50 12.12.5 13. 系统时钟/8 14. 接收数据寄存器中有新数据 15. 写序列

三、简答题

1.(3 分) **双堆栈机制**: MSP(主堆栈指针) 用于复位和异常处理; PSP(进程堆栈指针) 供线程模式下的用户代码使用。RTOS 中每个任务使用自己的 PSP, 实现任务堆栈隔离。

2.(4 分) **输出比较 vs PWM**: 输出比较 = 单次事件 (匹配时电平变化/中断/DMA); PWM = 持续波形输出。配置差异: OCMODE(Toggle/Active/Inactive vs PWM1/PWM2)、CCR 含义 (比较阈值 vs 占空比)。OC 适合事件定时, PWM 适合功率控制。

3.(3 分) **扫描 vs 中断模式**: 扫描模式 = 一次触发依次转换所有规则通道; 中断模式 (DISCEN) = 每次触发只转换一个通道 (或指定个数), 需多次触发完成全部通道。

四、程序设计题

第 1 题 (12 分): 数码管动态扫描——TIM4 5ms 中断, 4 位轮扫。DisplayNumber 将数字分解为 4 位, 存入显示缓冲区。

第 2 题 (16 分): USART 环形缓冲区 + 字符串反转——环形缓冲接收, 检测“提取一行, 双指针反转后发送”。

第 3 题 (18 分): PWM 水泵流量控制—— $f = 100\text{Hz}$, 取 $PSC = 7199$, $ARR = 99$: $72\text{M}/(7200 \times 100) = 100\text{Hz}$ 。10 档 (每档 10%)。5 个 LED 二进制显示档位。

第 4 题 (14 分): 测速 + 调速综合
比例控制核心:

```
1  /* 主循环 */
2  while(1) {
3      uint16_t target_adc = ADC_Read_Target(); /* PA1电位器 */
4      uint16_t target_rpm = (uint32_t)target_adc * 3000 / 4095;
5      uint16_t actual_rpm = GetMotorRPM(); /* TIM2捕获测得的转速 */
6
7      int16_t error = target_rpm - actual_rpm;
8      int16_t duty = pid_output + error * 3 / 10; /* 简易比例控制 */
9      if(duty < 0) duty = 0;
10     if(duty > 999) duty = 999;
11     TIM_SetCompare1(TIM3, duty);
12 }
```

试卷 13 参考答案与解析

一、选择题

1.A(处理器主频/存储/外设受限) 2.A(特权级可访问所有资源) 3.D(JTAG 占用更多 IO) 4.A(PA0 和 PA1 同时输出高) 5.B(汽车电子局部互联) 6.A(基本定时器无输出比较) 7.A(电压超阈值产生中断) 8.B(同一编号只能选一个端口) 9.C(stm32f10x_hal.c 非标准库文件) 10.A(同步模式需 CK 时钟) 11.A(CNT 清零 +UEV) 12.A(上拉悬空 = 高电平) 13.A(计数器溢出) 14.C(启动文件不可随意修改) 15.A(调试阶段检查参数有效性)

二、填空题

1. 仿真器/JTAG/SWD 2.24/操作系统时基 3.GPIOA-G/USART1/SPI1/TIM1
4.RCC_APB2PeriphClockCmd() 5.TIM_ClearITPendingBit() 6.ADC_CR2 的 CONT 位 7.USART_ITConfig() 8.NVIC_Init() 9.0x22000000 10. 递增和递减 11. 延时法/多次采样判断法
12.PCLKx/PCLK2 13. 长 14.4096/0.81 15. 集中包含所有外设库头文件

三、简答题

1.(4 分)PWM 频率与分辨率制约: $f_{PWM} = f_{CK} / [(PSC + 1)(ARR + 1)]$, 分辨率 $= \log_2(ARR + 1)$ 。 f_{CK} 固定时, ARR 增大 $\rightarrow$ 分辨率增加 $\rightarrow f_{PWM}$ 降低, 反之亦然。LED 调光 (高频 $>1\text{kHz}$ 防闪烁, 分辨率 >8 位); 舵机控制 (低频 50Hz , 高分辨率 >12 位)。
2.(3 分) 奇偶校验: 发送端根据数据位中 1 的个数置校验位 (奇校验 = 使总 1 数为奇)。接收端验证, 不匹配则置 PE 标志。局限性: 仅能检测奇数个位错误, 偶数个位同时翻转时漏检。
3.(3 分) 优先级分组影响: 组 0=0 抢占 +4 响应 (16 级响应), 组 2=2 抢占 (0-3)+2 响应 (0-3), 组 4=4 抢占 (0-15)+0 响应。抢占位数越多, 中断嵌套越灵活, 但响应优先级区分度越小。

四、程序设计题

第 1 题 (12 分): 交通灯模拟——TIM2 中断计时, 状态机 (红灯 10s $\rightarrow$ 绿灯 7s $\rightarrow$ 黄灯闪 3s $\rightarrow$ 循环)。黄灯闪烁: 0.5s 亮/灭交替 3 次 = 3s。
第 2 题 (16 分): ADC 阈值报警——上下阈值比较: $V > 2.5\text{V} \rightarrow \text{PC8}$ 亮; $V < 0.5\text{V} \rightarrow \text{PC9}$ 亮; 正常范围两 LED 灭。
第 3 题 (18 分): PWM 可编程波形——4 种模式通过按键循环切换。呼吸灯模式使用 TIM2 10ms 中断渐变。模式切换时重配置 PSC/ARR。
第 4 题 (14 分): 温湿度闭环系统
命令解析和自动模式代码:

```
1 volatile uint8_t auto_mode = 1;
2
3 float GetTemp(void) {
4     uint16_t adc = ADC_Read_CH1();
5     float v = adc * 3.3f / 4095.0f;
6     return v * 100.0f; /* LM35: 10mV/°C */
7 }
8
9 void AutoControl(void) {
10     float t = GetTemp();
11     uint16_t heater_duty, fan_duty;
12     if(t < 20.0f) {
13         heater_duty = 1000; fan_duty = 0;
14     } else if(t > 30.0f) {
15         heater_duty = 0; fan_duty = 1000;
16     } else {
17         heater_duty = (uint16_t)(1000.0f * (30.0f - t) / 10.0f);
18         fan_duty = 1000 - heater_duty;
19     }
20     TIM_SetCompare1(TIM3, heater_duty);
21     TIM_SetCompare2(TIM3, fan_duty);
22 }
```

试卷 14 参考答案与解析

一、选择题

1.B(ARM=IP 授权公司) 2.A(CONTROL[1]=SPSEL) 3.C(HSI/2 或 HSE 或 HSE/2)
4.A(Mode=AIN) 5.A(DR 读 =RDR 写 =TDR) 6.B(16 位) 7.D(VDDA 和 VSSA) 8.D(不需要 handler 函数名, 那是用户实现) 9.B(OCxPE 使能后写 CCR 在 UEV 时生效) 10.B(非 FT 引脚 VDD+0.3V) 11.B(旧数据未读新数据到) 12.B(2 分频) 13.A(防止上下管同时导通) 14.B(采样 + 转换时间) 15.A(获取各总线时钟频率)

二、填空题

1.16/R13/R14/R15 2.HCLK/PCLK1/PCLK2 3.4/输入输出和速率/具体模式 4. 发送移位寄存器/接收移位寄存器 5. 中断 DMA 请求/输出引脚动作 6. 复位校准/开始校准 7.PVD 输出/RTC 闹钟/USB 唤醒 8.SRAM 区/外设区 9.USART_DR/自动清除此标志 10.1000/10 11.10~20/按键抖动时间 (5~10ms) 12.1/2/4/8 13.5V/3.3V 到 5V 的电平转换 14. 外部信号源内阻/采样时间/长 15. 内核访问/外设控制

三、简答题

1.(4 分) 时钟树: HSE(8MHz)→PLL(×9)→PLLCLK(72MHz)→SW→SYSCLK(72MHz)→AHB 预分频 (/1)→HCLK(72MHz)。APB1 预分频 (/2)→PCLK1(36MHz), APB2 预分频 (/1)→PCLK2(72MHz)。
2.(3 分)SNR/ENOB/分辨率: 分辨率 = 理论位数 (如 12 位); SNR= 信号功率与噪声功率比 (dB), $SNR \approx 6.02N + 1.76$ (理想 N 位); ENOB= 有效位数 = $(SNR - 1.76)/6.02$, 反映实际可用精度, 因噪声和谐波失真通常低于分辨率。
3.(3 分) 中断延迟: 指从中断请求到执行 ISR 第一条指令的时间。Cortex-M3 措施: 硬件自动压栈 (减少软件开销)、向量表直接取 ISR 地址 (跳转无需查询)、尾链优化 (连续中断不重复压栈出栈)。

四、程序设计题

第 1 题 (12 分): 频率计——PA0 EXTI 计数 (或 TIM 输入捕获), TIM2 1s 闸门 (ARR=9999, PSC=7199)。ISR 中计算频率 $f = \text{计数值}$ 。8 个 LED 二进制显示。
第 2 题 (16 分): Modbus 简易从机——USART1 9600bps 接收。TIM4 3.6ms 帧间隔检测 (3.5 字符时间 = $3.5 \times 10 / 9600$ 3.65ms)。地址匹配后处理命令。
第 3 题 (18 分): PWM 步进电机模拟——四路 PWM(CH1~CH4), 频率 500Hz, 占空比 50%。各路 CCR 初值偏移以产生 90° 相位差 (CH1=0, CH2=ARR/4, CH3=ARR/2, CH4=3×ARR/4)。SetSpeed 改 PSC 变频率。
第 4 题 (14 分): 双机 USART 通信——设备 A: ADC 采样 →CCR 计算 →USART 发送“DXXXX”帧。设备 B: USART 接收 →解析 →设置 CCR。帧格式“D0000”~“D1000”。

试卷 15 参考答案与解析

一、选择题

1.B(非易失性存储器中的软件) 2.C(LR=0xFFFFFFFF= 返回线程模式 PSP) 3.B(备份域供电)
4.D(模拟输入禁止数字功能) 5.B(地址识别) 6.A(多次溢出后产生 UEV) 7.B(软件 + 多种外部触发)
8.A(节约向量表空间) 9.B(5V 容忍引脚) 10.C(先上后下) 11.D(三种方式均可) 12.A(VDDA 可以和
VDD 共用 3.3V 但需滤波) 13.A(抢占 2bit+ 响应 2bit) 14.A(滤除宽度 $<N \times t_{DTS}$ 的毛刺) 15.A(读取
IDR 实际电平)

二、填空题

1. 成本/功耗/体积/实时性/可靠性 2. 系统异常/外部中断 3. 外部直接输入时钟信号 4. 外设模块
5. 冻结/有效/无效/翻转/PWM1/PWM2 6. 逐次逼近转换所需固定周期数 7. USART_SR 标志位
8. AFIO_EXTICR 9. SysTick_CTRL/CLKSOURCE 10. ARR/影子寄存器 11. 别名区一个 32 位字/原
子性 12.4 13. 高级定时器/死区控制和刹车功能 14. 两个或以上 ADC 15. 中断服务函数 (ISR)

三、简答题

1.(4 分)UEV+DMA 更新 CCR: 定时器溢出 →UEV→DMA 请求 (DMA/DPY)→DMA 自动将
内存中的下一个 CCR 值搬运到定时器的 CCR 寄存器。这样可产生任意波形序列而无需 CPU 干预。
2.(3 分)中断 vs DMA: 中断方式 = 每个字节触发 CPU 处理 (适合低速/少量数据); DMA= 硬件
自动搬运数据 (适合高速/大量数据, 不占用 CPU)。ADC 多通道扫描 +DMA= 无需 CPU 即可完成
全部通道采集。
3.(3 分)Fault 异常: HardFault(通用错误, 其他 fault 未使能时触发); MemManage(MPU 违规, 非
法内存访问); BusFault(总线错误, 如访问无效地址); UsageFault(未定义指令, 除零, 非对齐访问等)。

四、程序设计题

第 1 题 (12 分): GPIO 中断 +LED 二进制计数——PA0(+)/PA1(-), 8 位计数器, LED 二进制
显示, TIM4 消抖。
第 2 题 (16 分): PWM 舵机无线控制——PSC=71, ARR=19999(50Hz)。0°=CCR500,
90°=CCR1500, 180°=CCR2500。USART 接收“Axxx”解析角度。
第 3 题 (18 分): 多路 PWM 独立输出——三路独立定时器 (TIM2/TIM3/TIM4), 各自配置
PSC/ARR/CCR。TIM2:1ms 时基, TIM3:2kHz/30%, TIM4:1kHz/70%。
第 4 题 (14 分): 多传感器数据采集与控制系统
控制策略核心逻辑:

```
1 volatile uint8_t sys_on = 0; /* 采集开关 */
2
3 void EXTI0_IRQHandler(void) {
4     /* 20ms消抖后切换开关状态 */
5     sys_on = !sys_on;
6     EXTI_ClearITPendingBit(EXTI_Line0);
7 }
8
9 int main(void) {
10     uint16_t adc_t, adc_l;
11     /* ...初始化... */
12     while(1) {
13         if(sys_on) {
14             adc_t = ADC_Read_CH1(); /* 温度 */
15             adc_l = ADC_Read_CH2(); /* 光照 */
16             /* 温度低→加热PWM大(反比), 光照低→补光PWM大(反比) */
17             TIM_SetCompare1(TIM3, 999 - (uint32_t)adc_t * 999 / 4095);
18             TIM_SetCompare2(TIM3, 999 - (uint32_t)adc_l * 999 / 4095);
19         } else {
20             TIM_SetCompare1(TIM3, 0);
21             TIM_SetCompare2(TIM3, 0);
22         }
23         /* USART 1s上报逻辑... */
24     }
25 }
```

试卷 16 参考答案与解析

一、选择题

1.B(集成 CPU+ 存储器 + 外设的单芯片) 2.B(EXC_RETURN 触发硬件出栈) 3.B(0x20000000)
4.B(外设模块) 5.B(LSB 低位先发) 6.A(不分频,PSC+1=1) 7.B(扫描模式 DMA 保存各通道) 8.D(唤醒 CPU 或触发外设) 9.B(需调用 GPIO_Init 配置) 10.B(清 0 重新计数) 11.A(OCPolarity 决定) 12.A(软件触发启动规则组转换) 13.B(模拟输入功耗最低, 施密特触发器关闭) 14.B(屏蔽优先级 某值的中断)
15.B(启动停止计数器)

二、填空题

1. 嵌入式处理器/存储器/I/O 接口/电源管理/嵌入式软件 2.0x00000000/VTOR
3.0x0801FFFF 4. 禁止/使能 5. 有效电平的定义/高电平 6. 二分搜索 7. 汽车电子/红外通信
8.AFIO_EXTICR/EXTI/NVIC 9.HSI(8MHz) 10. 更新事件/比较匹配 11.US-ART_SR/USART_DR
12.CCR/(ARR+1) 13.ADC_SoftwareStartConvCmd()/一次 14. 以太网唤醒事件 15.startup_xxx_ld/_md/_hd/_xl

三、简答题

1.(3 分) 位带操作优势: 原子操作 (避免 read-modify-write 竞态), 简化代码 (无需掩码)。典型场景: GPIO 引脚独立控制、状态标志位操作。
2.(4 分) 频率 vs 分辨率: f 与 $\log_2(ARR + 1)$ 成反比。折中策略: 舵机控制 $\rightarrow$ 低频率 (50Hz) 换取高分辨率 (ARR 20000, >14 位); LED 调光 $\rightarrow$ 高频 (>1kHz) 防止闪烁, ARR 1000(10 位) 可接受。
3.(3 分)ISR 编写原则: 尽量简短 (快速返回); 避免阻塞等待 (如 while 等标志); 注意共享数据保护 (volatile/临界区); 正确清除中断标志; 避免在 ISR 中调用可能重入的函数。

四、程序设计题

第 1 题 (12 分): 跑马灯——TIM4 1s 中断 (PSC=7199,ARR=9999)。LED 索引递增/递减往复。
第 2 题 (16 分): ADC 调光台灯——ADC1 读电位器 $\rightarrow$ CCR 映射 (0~999)。PA0 EXTI 切换手动/自动模式。自动模式 =CCR=500(50%)。
第 3 题 (18 分): PWM 蜂鸣器音乐——PlayTone(freq) 函数动态计算 PSC 和 ARR。TIM2 500ms 节拍中断, 按乐谱数组索引依次播放。
第 4 题 (14 分): 双传感器 +PWM 双路 +USART 监控
命令处理框架:


```
1 volatile uint8_t manual_mode = 0; /* 0=ADC自动, 1=手动 */
2
3 void ProcessCmd(void) {
4     if(cmd[0] == 'A') {
5         manual_mode = 0; /* 切回自动 */
6     } else if(cmd[0] == 'M' && cmd[1] == '1') {
7         uint16_t v = (cmd[2]-'0')*100 + (cmd[3]-'0')*10 + (cmd[4]-'0');
8         TIM_SetCompare1(TIM3, v);
9         manual_mode = 1;
10    } /* M2 类似... */
11 }
12
13 /* 主循环 */
14 while(1) {
15     if(!manual_mode) {
16         adc1 = ADC_Read_CH1(); adc2 = ADC_Read_CH2();
17         TIM_SetCompare1(TIM3, (uint32_t)adc1 * 999 / 4095);
18         TIM_SetCompare2(TIM3, (uint32_t)adc2 * 999 / 4095);
19     }
20     /* 500ms上报... */
21 }
```

试卷 17 参考答案与解析

一、选择题

1.B(软硬件并行 + 功能划分) 2.A(硬件自动压栈) 3.A(等待 PLLRDY) 4.A(配置锁定直到复位)
5.C(USART 只有一个全局中断向量) 6.B(CCR=16 位) 7.B(温度 ↑ 电压 ↓) 8.A(事件不产生中断)
9.C(SWD 速度与 JTAG 相近) 10.B(APB1 预分频 1 时定时器时钟 =APB1×2) 11.A(CNT 清 0)
12.A(返回 IDR 全 16 位) 13.A(产生精确脉冲数) 14.B(最优值, 兼顾噪声和响应) 15.A(低 8 位输出高)

二、填空题

1.32 2.SWD/JTAG/调试端口 (DP)/访问端口 (AP) 3.RCC_APB2Periph_GPIOx/RCC_APB2Periph_USART1
4. 关闭/功耗 5. 无效 6. 软件触发/外部事件 7. 起始位/数据位中心 8.AFIO/GPIO_EXTI_LineConfig()
9.32 位字/原子位操作 10. 高级定时器 (TIM1/TIM8) 11. 无效电平 (低) 12.1 13.72/36/72 14.3 15. 使
能时钟 → 配置结构体 → Init → 使能中断 NVIC → 使能外设

三、简答题

1.(3 分)SysTick: 24 位递减计数器, 时钟源可配 (HCLK 或 HCLK/8), 适合产生 RTOS 节拍 (如 1ms) 或精确延时。与通用定时器的区别: SysTick 是内核集成 (所有 Cortex-M3 都有), 通用定时器是芯片外设 (不同型号可能有差异)。
2.(3 分)UIE vs CCxIE 标志: UIE 在 CNT 溢出时硬件置位 UIF, 需读 SR 后写 0 清除 (或直接软件清除); CCxIE 在 CNT=CCR 时硬件置位 CCxIF, 清除方式同 UIF。
3.(4 分) 查询/中断/DMA 对比: 查询 =CPU 持续轮询标志 (简单但效率低, 适合单任务); 中断 = 事件触发 CPU 处理 (效率高, 但频繁中断增加上下文切换开销, 适合中低速); DMA = 硬件自动搬运 (效率最高, CPU 几乎不参与, 适合高速大数据量)。

四、程序设计题

第 1 题 (12 分): 长按/短按识别——TIM4 10ms 中断做按键扫描。按下计时: <100 次 (1s)= 短按 → LED 翻转; 100 次 = 长按 → LED 快闪 3 次。
第 2 题 (16 分): USART 超时接收——每收到一个字节重置 TIM4 计数器 (5ms)。TIM4 超时中断标记数据包完成。主循环处理完整数据包。
第 3 题 (18 分): PWM 呼吸灯 + 频率可调——TIM4 10ms 时基用于呼吸渐变和按键扫描。PA0/PA1 调呼吸速度 (改变延时), PA2 切换 PWM 频率 (PSC 重配置)。
第 4 题 (14 分): 智能鱼缸控制器
温控和光照控制:

```
1 float GetTemp(void) {
2     uint16_t adc = ADC_Read_CH1();
3     float v = adc * 3.3f / 4095.0f;
4     return 25.0f + (1.65f - v) / 0.02f; /* NTC公式 */
5 }
6
7 void ControlLoop(void) {
8     float t = GetTemp();
9     uint16_t light_adc = ADC_Read_CH2();
10    /* 加热控制(目标25°C) */
11    if(t < 25.0f) {
12        uint16_t heater = (uint16_t)((25.0f - t) * 200);
13        if(heater > 1000) heater = 1000;
14        TIM_SetCompare1(TIM3, heater);
15    } else TIM_SetCompare1(TIM3, 0);
16    /* 补光控制(光照低-占空比大) */
17    uint16_t light_pwm = 999 - (uint32_t)light_adc * 999 / 4095;
18    TIM_SetCompare2(TIM3, light_pwm);
19 }
20
21 /* EXTI0 缺水报警 */
22 void EXTI0_IRQHandler(void) {
23     /* 消抖后: LED快闪, 所有输出关闭 */
24     GPIO_ResetBits(GPIOC, GPIO_Pin_13); /* 蜂鸣器报警 */
25     TIM_SetCompare1(TIM3, 0);
26     TIM_SetCompare2(TIM3, 0);
27     EXTI_ClearITPendingBit(EXTI_Line0);
28 }
```

试卷 18 参考答案与解析

一、选择题

1.D(桌面 GUI 库非交叉工具链) 2.C(0xFFFFFFFF= 返回处理模式 MSP) 3.B(HSI 默认开启并作为系统时钟) 4.A(输出驱动器压摆率) 5.B($1+8+1+1=11\text{bit}$) 6.C(基本定时器无输出比较/PWM) 7.A(连续扫描 +DMA 循环 = 完整采集) 8.A(已触发中断请求) 9.B(Cmd 结尾 = 使能/禁止) 10.A(当前 CNT 值) 11.B(定时器所在总线时钟, 经 APB 预分频修正) 12.B(等待 TXE= 可以发送下一字节) 13.B(并非所有外设的所有功能都可重映射) 14.A(冻结 = 匹配时不改变输出) 15.B(写 1 的引脚输出低电平)

二、填空题

1. 程序指令和数据存储器合并/同一 2.4/高 4 位 3.PLLMUL/PLLDIV 倍频系数 4. 不确定 5.GPIO 模式 (AF_PP)/时基初始化/输出比较初始化/使能定时器 6.(V25-Vsense)/Avg_Slope+25 7. 地址数据帧 8.AFIO_EXTICR/4 9.10ms 10. 定时器和 NVIC 初始化/main 函数 11. 不断改变 CCR/占空比 12.8 位精度 13.uint16_t/低 8 位 14.APB2/APB1 15.Flash/SRAM/SRAM

三、简答题

1.(3 分)AHB/APB1/APB2: AHB= 高速总线 (CPU/内存/DMA,72MHz); APB1= 低速外设总线 (TIM2-TIM7,USART2-UART5,SPI2/I2C, 最高 36MHz); APB2= 高速外设总线 (GPIO,TIM1,USART1,SPI1,ADC, 最高 72MHz)。
2.(3 分) 输入捕获 vs PWM 输入模式: 普通输入捕获 = 单通道捕获边沿; PWM 输入模式 = 双通道联合 (一个捕获上升沿, 另一个捕获下降沿), 硬件自动完成, 单次配置即可测得频率和占空比。
3.(4 分)ADC 精度影响因素: 参考电压稳定性 (VREF 噪声直接引入误差); ADCCLK 频率 (>14MHz 精度下降); 采样时间 (太短 → 保持电容充电不充分); 信号源输出阻抗 (高阻抗需长采样时间); PCB 布局 (数字噪声耦合到模拟信号); 温度 (温漂影响)。

四、程序设计题

第 1 题 (12 分): 串口控制台——USART1 接收中断, 回车触发命令匹配 (不区分大小写), strcmp 比较。
第 2 题 (16 分): 定时器级联长延时——TIM4 1ms 中断 (PSC=71,ARR=999), ISR 中软件计数。计数到 10000=10s → 翻转 LED。
第 3 题 (18 分): PWM 电磁铁控制—— $f = 20\text{kHz}$, 取 $PSC = 71, ARR = 49$: $72\text{M}/(72 \times 50) = 20\text{kHz}$ 。但 ARR 太小 → 改为 $PSC = 35, ARR = 99$: $72\text{M}/(36 \times 100) = 20\text{kHz}$ (分辨率 100 级)。更好方案: $PSC = 3, ARR = 899$: $72\text{M}/(4 \times 900) = 20\text{kHz}$ (分辨率 900 级)。
第 4 题 (14 分): 多通道 ADC+DMA——ADC1 四通道扫描 +DMA 循环模式 (4 个 uint16_t 数组)。DMA_InitStructure.DMA_BufferSize = 4; DMA_InitStructure.DMA_Mode = DMA_Mode_Circular。

试卷 19 参考答案与解析

一、选择题

1.B(智能手环 = 典型嵌入式) 2.B(屏蔽除 NMI 外的所有, 含 HardFault) 3.B(FLASH_ACR 的 LATENCY 位) 4.B(推挽输出 P-MOS 导通拉电流最强) 5.B(一个帧周期内无新起始位) 6.B(配置各通道捕获/比较) 7.A(ADC_HTR= 高阈值) 8.B(同编号只能选一个端口) 9.B(启动文件 Reset_Handler) 10.C(CNT 变为 0 而非 ARR) 11.A(PWM1 和 PWM2 有效电平逻辑相反) 12.B(stm32f10x_conf.h) 13.C(6 分频 =12MHz, 在 14MHz 内) 14.A(约 40kΩ) 15.A(PCLK2/6 作为 ADC 时钟)

二、填空题

1. 同一个时钟周期/指令/数据 2.APSR/IPSR/EPSR 3.GPIO 引脚/PVD 输出 4.GPIO_Pin_0/GPIO_Pin_15/按位或 5.16/65536 6.ADC_DR 7. 高/低 8. 中断/事件/NVIC/其他外设 9.TRGO(触发输出) 10. 有效 11.main.c/stm32f10x_it.c 12.EXTI_GetITStatus() 13.&ADC1->DR/外设到内存 14. $D \times 3.3/4096/D \times 0.806$ 15. 时钟系统配置/RCC_APB2PeriphClockCmd

三、简答题

1.(3 分)AFIO 主要功能: EXTI 输入线的 GPIO 端口选择 (GPIO_EXTILineConfig); 外设引脚重映射 (USART/SPI/TIM 等的 Remap); JTAG/SWD 引脚配置。
2.(3 分)舵机 PWM 映射: $f = 50\text{Hz} \rightarrow$ 周期 = 20000 计数 (1MHz 时基)。CCR = $500 + \text{angle} \times 2000/180$ 。0°→CCR=500(0.5ms), 90°→CCR=1500(1.5ms), 180°→CCR=2500(2.5ms)。
3.(4 分)波特率误差: 晶振频率不匹配 → 波特率分频值非整数 → 累积误差。一般要求总误差 <2%(含收发双方)。可通过选择波特率 (如 9600→BRR 为整数)、使用合适晶振 (如 11.0592MHz 产生精确波特率) 来减小误差。一个数据帧需在 10~11 位内所有采样点不偏移超过半个位。

四、程序设计题

第 1 题 (12 分): 多按键 LED 控制——4 个按键独立功能: 左移/右移/全亮全灭切换/自动模式。TIM4 10ms 扫描。
第 2 题 (16 分): PWM 信号发生器——USART 接收命令: ‘F’设置频率 (改 PSC), ‘D’设置占空比 (改 CCR), ‘S’停止, ‘G’恢复。PSC 公式: $PSC = 72000/f_{Hz} - 1$ (ARR=999)。
第 3 题 (18 分): PWM H 桥电机控制—— $f = 20\text{kHz}$, 取 $PSC = 71, ARR = 49$ (分辨率低)。改进: $PSC = 0, ARR = 3599$ (分辨率 = 3600 级, $f = 72\text{M}/3600 = 20\text{kHz}$)。MotorControl(speed>0→CH1=speed%+CH2=0, speed<0→CH1=0+CH2=|speed|%, speed=0→CCR1=CCR2=0)。
第 4 题 (14 分): 环境监测站
温控和风扇控制:

```
1 void AutoControl(void) {  
2     float t = GetTemp(); float h = GetHumidity();  
3     uint16_t heater = 0, fan = 0;  
4  
5     if(t < 20.0f) {  
6         heater = 1000; fan = 0;  
7     } else if(t <= 30.0f) {  
8         float ratio = (t - 20.0f) / 10.0f; /* 0~1 */  
9         heater = (uint16_t)(1000.0f * (1.0f - ratio));  
10        fan = (uint16_t)(1000.0f * ratio);  
11    } else {  
12        heater = 0; fan = 1000;  
13    }  
14  
15    TIM_SetCompare1(TIM3, heater);  
16    TIM_SetCompare2(TIM3, fan);  
17 }
```

试卷 20 参考答案与解析

一、选择题

1.C(并非所有嵌入式系统都需要 OS) 2.A(屏蔽除 NMI+HardFault 外的可屏蔽中断) 3.A(默认组 0=0 位抢占 +4 位响应) 4.B(填充结构体默认值) 5.A(清除标志或读 DR 自动清除) 6.C(ETR 不能配置 GPIO 模式,GPIO 需单独配) 7.D(左右对齐精度相同, 仅读取方式不同) 8.A(EXTI_SWIER) 9.A(调试验证定时器行为) 10.A($ARR=19999 \rightarrow 20000$ 级 $\rightarrow$ 约 15 位) 11.B(迟滞特性提高抗干扰) 12.A(TXE 适合连续发送,TC 适合确认最后字节) 13.A(使能 CCR1 预装载) 14.B(VDDA 接 3.3V+ 滤波,VSSA 接 GND) 15.D(A 和 B 均推荐)

二、填空题

1. 交叉编译/宿主机 (PC)/目标机 (嵌入式设备) 2. 特权/用户/特权 3.25/150
4. 上拉电阻/目标高电平电压 5. 无效 6.ADC_SQRx 7.TDR(发送)/RDR(接收)
8.EXTI_Trigger_Rising_Falling/RTSR/FTSR 9.SystemInit()/系统时钟 10. 极性选择/输出使能
11. 允许发送/暂停发送 12.25 13. 页 (Page)/页/全片 14.4/数据寄存器 (ADC_JDRx) 15. 外设缩写 _ 功能描述

三、简答题

1.(4 分) 上电 $\rightarrow$ main() 全过程: 上电复位 $\rightarrow$ 从 0x00000004 读取 Reset_Handler 地址 $\rightarrow$ 执行 Reset_Handler(启动文件): 初始化 MSP、调用 SystemInit() 配置时钟 (默认 HSI $\rightarrow$ PLL $\rightarrow$ 72MHz) $\rightarrow$ 初始化.data 段 (从 Flash 复制到 SRAM)、清零.bss 段 $\rightarrow$ 调用 __libc_init_array(全局构造函数) $\rightarrow$ 跳转 main()。
2.(3 分) 舵机 PSC/ARR 推荐: $f_{CK} = 72\text{MHz}$, $f_{PWM} = 50\text{Hz}$ 。取 $ARR = 19999$ (20000 级分辨率, >14 位) 则 $PSC = 71$: $72\text{M}/72/20000 = 50\text{Hz}$ 。舵机需要高占空比精度 ($0^\circ\sim 180^\circ$ 对应 2ms 脉宽范围), $ARR=19999$ 时 CCR 分辨率约 1 s(1 计数 =1 s)。
3.(3 分) 课程知识框架: 嵌入式系统基础 (定义/特点/MCU vs MPU/哈佛结构); Cortex-M3 体系 (内核/寄存器/NVIC/堆栈/异常); STM32 最小系统 (复位/时钟/启动/下载); GPIO(模式/寄存器/驱动); EXTI(原理/配置/优先级); USART(帧格式/波特率/中断); 通用定时器 (时基/PWM/输入捕获); ADC(原理/分辨率/扫描/DMA)。

四、程序设计题

第 1 题 (12 分): GPIO+USART 交互——USART 接收'1'-'4' 切换对应 LED(奇数次亮偶数次灭)。发送'S' 返回 4 个 LED 状态字符串。
第 2 题 (16 分): 定时器 +ADC 数据采集——TIM2 100ms 中断启动 ADC 转换, 采集 10 次取均值。PC13 LED 采集期间快闪。
第 3 题 (18 分): PWM 多模式调度——TIM2 10ms 时基, 软件计时实现 5s 模式轮换。3 种模式

(PWM 参数各不相同 → 重配 PSC/ARR/CCR)。USART 上报当前模式和剩余时间。

第 4 题 (14 分): 智能温室控制器

完整系统框架:


```

1  /* ===== 系统软件架构(文字框图) =====
2  * main():
3  *   └─ 外设初始化(ADC/TIM3/TIM4/EXTI/USART1/GPIO)
4  *   └─ while(1):
5  *     └─ 读取ADC1双通道(土壤湿度+光照)
6  *     └─ 控制决策:
7  *       └─ 湿度<阈值 → 灌溉PWM(反比映射)
8  *       └─ 光照<阈值 → 补光PWM(反比映射)
9  *       └─ 检查灌溉计时标志 → 超时关闭灌溉
10  *       └─ 检查5s上报标志 → USART发送数据
11  * TIM4_IRQHandler:
12  *   └─ 系统LED闪烁(500ms)
13  *   └─ 灌溉计时(5s倒计时)
14  * EXTI0_IRQHandler:
15  *   └─ 手动灌溉: 设置灌溉标志, 启动5s计时, PWM 100%
16  * USART1_IRQHandler:
17  *   └─ 接收数据(备用扩展)
18  * ===== */
19
20 volatile uint8_t irrigate_manual = 0, irrigate_flag = 0;
21 volatile uint16_t irrigate_timer = 0;
22 volatile uint32_t sys_ticks = 0; /* TIM4 10ms计数 */
23
24 void TIM4_IRQHandler(void) {
25     if(TIM_GetITStatus(TIM4, TIM_IT_Update) != RESET) {
26         TIM_ClearITPendingBit(TIM4, TIM_IT_Update);
27         sys_ticks++;
28
29         /* 系统LED 500ms闪烁 */
30         if(sys_ticks % 50 == 0)
31             GPIO_WriteBit(GPIOC, GPIO_Pin_13,
32                 (BitAction)!GPIO_ReadOutputDataBit(GPIOC, GPIO_Pin_13));
33
34         /* 灌溉计时 */
35         if(irrigate_flag) {
36             irrigate_timer++;
37             if(irrigate_timer >= 500) { /* 5s到 */
38                 irrigate_flag = 0;
39                 TIM_SetCompare1(TIM3, 0); /* 关闭灌溉PWM */
40                 GPIO_SetBits(GPIOC, GPIO_Pin_8); /* 灌溉指示LED灭 */
41             }
42         }
43     }
44 }
45
46 void EXTI0_IRQHandler(void) {
47     /* 20ms消抖后... */
48     irrigate_manual = 1;
49     irrigate_flag = 1;
50     irrigate_timer = 0;

```

系统设计要点：

1. 灌溉和补光都采用反比映射：传感器值越大 (湿度/光照越足)→PWM 占空比越小。
2. 手动灌溉通过 EXTI 触发，具有最高优先级，并设置 5 秒定时器自动关闭。
3. 系统 LED(PC13) 作为心跳指示 (500ms 闪烁)。
4. 灌溉中 (PC8) 和补光中 (PC9) 两个状态 LED 实时显示执行器状态。
5. USART 每 5 秒上报一次数据用于远程监控。
6. TIM4 10ms 既是系统时基，也是 LED 闪烁和灌溉计时的基础。

附录：STM32 常用计算公式速查

1. 定时器计数频率: $f_{CK_CNT} = \frac{f_{CK_PSC}}{PSC + 1}$
2. 定时器溢出频率: $f_{OV} = \frac{f_{CK_CNT}}{ARR + 1} = \frac{f_{CK_PSC}}{(PSC + 1)(ARR + 1)}$
3. PWM 频率 (边沿对齐): $f_{PWM} = \frac{f_{CK_PSC}}{(PSC + 1)(ARR + 1)}$
4. PWM 占空比 (PWM1, 有效电平高): $Duty = \frac{CCR}{ARR + 1} \times 100\%$
5. PWM 分辨率 (位数): $R = \log_2(ARR + 1)$
6. ADC 转换电压: $V_{in} = \frac{ADC_Value}{2^N} \times V_{ref}$ (N 为 ADC 位数)
7. USART 波特率 (16 倍过采样): $BRR = \frac{f_{CK}}{16 \times BaudRate}$
8. APB1 定时器时钟修正: APB1 预分频 $\neq 1$ 时, $f_{TIM} = 2 \times f_{PCLK1}$; APB1 预分频 $= 1$ 时, $f_{TIM} = f_{PCLK1}$ 。
9. STM32F103 地址映射:
 - Flash: 0x08000000
 - SRAM: 0x20000000
 - 外设: 0x40000000
 - 位带别名区 (SRAM): 0x22000000
 - 位带别名区 (外设): 0x42000000
10. GPIO 引脚最大电流: 单个引脚灌/拉电流 $\leq 25\text{mA}$; 所有 GPIO 总电流 $\leq 150\text{mA}$ 。

——《ARM Cortex-M3 嵌入式系统》期末考试题库（教师版）完——

共计 20 套完整试卷, 覆盖全部课程知识点

使用标准库 (StdPeriph Library) 编写全部编程题参考答案